



APX

APX

Recursos

Hay seis tipos de recurso disponibles para crear en APX: Unidad de despliegue online, unidad de despliegue batch, librería, DTO, shell script y estáticos.

Antes de seguir hay que aclarar que a lo largo de esta guía mencionaremos **Recurso** para referirnos a los elementos que nos aparecen listados en la Consola Ether al entrar en una UUAA. Cada recurso tiene un (solamente uno) repositorio de código asociado. Diremos **Componente** para referirnos a los binarios (*.jar) o ficheros *.sh *.sql que forman los recursos. Por ejemplo, diremos que un *recurso* de tipo Unidad de despliegue Online llamado 'mi_recurso' contiene cuatro *componentes*, dos transacciones, una librería y un DTO.

Unidad de despliegue

Una unidad de despliegue es una agrupación de componentes APX (transacción, librería, dto) que siguen el mismo ciclo de vida y se despliegan en bloque. Puede contener de uno a muchos componentes APX. Cada aplicación puede tener las UD que quiera. Se puede tener una UD por componente APX. Una UD se puede desplegar tanto en máquinas virtuales como contenedores con el mismo código (siempre que los componentes puedan constituir un contenedor funcional).

La unidad de despliegue sirve para dar una solución de agrupación de componentes APX en un mismo repo. No es obligatorio juntar ninguno de nuestros componentes. De hecho, perfectamente podemos tener una UD por cada una de nuestras transacciones, sin más componentes dentro.



Existen dos tipos de unidades de despliegue, las de online y las de batch. Las de online pueden contener transacciones, dtos y librerías, y las de batch jobs, dtos y librerías. Una particularidad de las de Batch es que solo vamos a poder tener un job por unidad de despliegue.

¿Cómo va a afectar a los repositorios estas agrupaciones? Donde antes (Stash) teníamos un repo que albergaba una transacción, ahora nos encontraremos una UD con esa transacción dentro. Donde antes teníamos un repo con un job, ahora nos encontraremos una UD con ese job dentro. En el caso que queramos, podremos añadir más artefactos APX a nuestras UD.

Estructura

La estructura de una unidad de despliegue es la siguiente:

Source	Description
 artifact	
 doc	
 .gitignore	Initial scaffolding
 Jenkinsfile	Initial scaffolding
 pom.xml	Initial scaffolding
 README.md	Initial scaffolding

En la raíz tenemos dos carpetas y cuatro archivos. La carpeta artifact, que va a contener todos los artefactos de APX de nuestra UD. La carpeta doc, que va a tener dentro un archivo para que los desarrolladores puedan documentar con detalle sus componentes. El fichero .gitignore, que usa git para poder ignorar cualquier fichero innecesario que se nos genere en local y no queramos subir (por ejemplo los ficheros temporales que pueda generar un IDE). El Jenkinsfile, ~~que se usa para configurar los jobs de jenkins que ejecutará nuestro recurso (el desarrollador no tiene que tocar nada de este fichero)~~ A partir de enero de 2021, los recursos se crearán con un fichero llamado 'apx.json' que sustituye al Jenkinsfile y cumple su misma funcionalidad. El pom.xml, pom padre del proyecto. Y por último un fichero README para cualquier uso que le quieran dar los dueños del repositorio.

Dentro de la carpeta artifact veremos una serie de carpetas según componentes:

Dentro de la carpeta artifact veremos una serie de carpetas según componentes:

Source	Description
..	
dtos	
libraries	
transactions	

Y dentro de cada una de estas ya estarán nuestros componentes:

develop | test_prod_marzo / artifact / transactions /

Source	Description
..	
HRCHT111-01-ES	
pom.xml	[jenkinsfile re

Librería

El recurso tipo librería es una agrupación de los componentes APX que forman la propia librería: interfaz e implementación. Antes, en Stash, teníamos dos repositorios separados con dos ciclos de vida para las interfaces e implementaciones. A partir de ahora van a compartir repositorio y ciclo de vida, y se desplegarán los dos a la vez. En el caso de la migración se juntarán en un mismo repo lo que haya en master de la interfaz y lo que haya en master de la implementación y ambas tendrán la versión de la interfaz.

Librería como recurso VS Librería en la UD

Como hemos visto anteriormente, podemos tener una librería tanto en una unidad de despliegue como en un recurso independiente. Esto nos da la ventaja de poder compartir una librería entre varias UD, o en caso que sea una librería compartida o 'de infraestructura', poder tener un ciclo de vida independiente para ella. Cuando una librería sólo va a ser usada dentro del contexto de una unidad de despliegue, recomendamos ponerla dentro, pero en caso de que sea susceptible de ser usada por más componentes, debería ir independiente. También tener en cuenta que una librería concreta (o cualquier otro componente de APX) solo puede existir en un único recurso.

Estructura

Una librería va a tener una estructura como esta:

Source	Description
 doc	
 HRCHR087	
 HRCHR087IMPL	
 .gitignore	Initial scaffolding
 Jenkinsfile	Initial scaffolding
 pom.xml	Initial scaffolding
 README.md	Initial scaffolding

La mayor diferencia con la de la UD es que en vez de la carpeta artifact, tiene directamente en la raíz las carpetas de la interfaz e implementación. El resto de componentes son iguales a los ya mencionados salvo el pom.xml que tiene algunos cambios para adaptarse a esta estructura.

Librerías multipaís

Existe otra posibilidad para este tipo de repositorios: las librerías multipaís. Este tipo de recurso es como una librería pero con varias implementaciones, una por cada país que se necesite. En caso de un recurso global con una interfaz global pero implementaciones específicas por país se puede usar esta solución. Más info [aquí](#).

Librerías multiruntime

Los recursos de tipo librería pueden ser utilizados indistintamente en APX Online o APX Batch, por este motivo, a partir del día **14/12/2022**, se dará la posibilidad de desplegar estos recursos de forma dual, es decir, en ambos runtimes simultáneamente. De esta manera, se facilita la operativa de despliegue para estos casos y se garantiza la alineación de versiones entre Online y Batch.

Con esta nueva funcionalidad, es importante tener en cuenta algunas consideraciones:

- El tipo de runtime (Online, Batch o Dual) deberá ser seleccionado en el primer despliegue en DEV que se realice de la librería, quedando establecido para los siguientes despliegues y sin posibilidad de editar.
- Con el despliegue multiruntime no se permitirá desplegar versiones diferentes de una misma librería en Online y Batch.
- Aunque el despliegue multiruntime implica intrínsecamente desplegar en dos runtimes diferentes, a nivel de Consola y Techstack, computará como un único despliegue y cualquier acción que se realice sobre el mismo: redeploy, rollback, etc,... aplicará, por tanto, en ambos runtimes.

Regularización de librerías antiguas

Aquellas librerías que fueron desplegadas antes de estar operativa esta nueva funcionalidad, deberán ser **regularizadas** por el developer cuando realice el **siguiente despliegue en DEV** seleccionando el modo de despliegue requerido. Pueden darse varios casos:

- Si nuestra librería estaba desplegada en un único runtime (Online o Batch) y queremos que así continúe, cuando se realice el siguiente despliegue en DEV, deberá seleccionarse la opción Online o Batch según corresponda.
 - Si nuestra librería estaba desplegada en un único runtime (Online o Batch) y queremos que a partir de ahora se despliegue en modo multiruntime, cuando se realice el siguiente despliegue en DEV, deberá seleccionarse la opción Dual (Online + Batch)
 - Si nuestra librería tenía versiones desplegadas tanto en Online como en Batch, cuando se realice el siguiente despliegue en DEV, deberá seleccionarse la opción Dual (Online + Batch).
- IMPORTANTE:** a partir de este momento, tanto en Online como en Batch quedará desplegada la misma versión de la librería.

Shell Script

El recurso de tipo Shell Script es simplemente una agrupación de ficheros .sh y/o .sql. Al no tratarse de componentes APX como tal (transacción, librería, job o DTO), como ocurre en el caso de una UD, tanto el versionado como el modelo de ramas que usaremos para este tipo de recursos será bastante más sencillo prescindiendo de las ramas RC y Release (veremos el detalle en el apartado [Modelo de ramas en Shell Script](#)).

Otros aspectos a tener en cuenta a la hora de trabajar con este tipo de recursos:

- Los ficheros agrupados en un mismo recurso compartirán ciclo de vida y por tanto desplegarán a la vez por los entornos.
- No pueden existir ficheros con el mismo nombre dentro de una misma aplicación
- Este tipo de recurso no pasa validación de Gobierno por lo que no será necesario indicar un ticket JIRA en el asistente de despliegue.

Estructura

Al crear un recurso de tipo Shell Script, se genera un repositorio en Bitbucket prácticamente vacío con la siguiente estructura:

Source	Description
 <code>apx.json</code>	Initial scaffolding
 <code>README.md</code>	Initial scaffolding

El desarrollador podrá subir a este repositorio los ficheros sh y/o sql que necesite teniendo en cuenta las siguientes consideraciones:

- No se permiten repositorios con ficheros con extensiones diferentes a .sql y .sh
- No pueden existir ficheros con el mismo nombre dentro de una misma aplicación ya que desplegarán en la misma ruta
- Si se organizan los ficheros en carpetas dentro del repositorio, esta organización no aplicará a la hora del despliegue ya que se todos los ficheros se copian en la misma ruta.

El recurso de tipo Statics es simplemente una agrupación de ficheros estáticos: plantillas, imágenes, hojas de estilo, etc,... Este tipo de ficheros son utilizados por los aplicativos que necesitan generar documentos con la utilidad *Document Creator* de APX.

Al no tratarse de un componente APX como tal (transacción, librería, job o DTO), como ocurre en el caso de una UD, tanto el versionado como el modelo de ramas que usaremos para este tipo de recursos será bastante más sencillo prescindiendo de las ramas RC y Release (veremos el detalle en el apartado [Modelo de ramas en Shell Script y Estáticos](#)).

Otros aspectos a tener en cuenta a la hora de trabajar con este tipo de recursos:

- Solo estará permitido el uso de un recurso de tipo Statics por aplicación. Todos los ficheros estáticos (css, plantillas, etc,...) de la aplicación deberán estar agrupados en el mismo recurso.
- Los ficheros agrupados en el recurso de tipo Statics compartirán ciclo de vida y por tanto desplegarán a la vez por los entornos.
- No pueden existir ficheros con el mismo nombre dentro de un mismo recurso
- Para los despliegues en Producción será necesario generar el tag a mano o desde Bitbucket.
- Este tipo de recurso no pasa validación de Gobierno por lo que no será necesario indicar un *ticket JIRA* en el asistente de despliegue.

Estructura

Al crear un recurso de tipo Statics, se genera un repositorio en Bitbucket con nombre **uuaa-statics** con la siguiente estructura:

Source

master ▾ ... qwpu-statics /	
Source	Description
statics-commons	
apx.json	Migration to Bitbucket from Stash
README.md	Migration to Bitbucket from Stash
README.md	

El desarrollador podrá subir a este repositorio los ficheros estáticos que necesite teniendo en cuenta las siguientes consideraciones:

- Existirá una carpeta obligatoria **statics-commons** donde se alojarán los estáticos de forma genérica. Esta carpeta se creará automáticamente en el repositorio al crear el recurso y contendrá las subcarpetas: **images, css, templates**

Source

 master ▾ ... | [qwpu-statics](#) / **statics-commons** /

Source

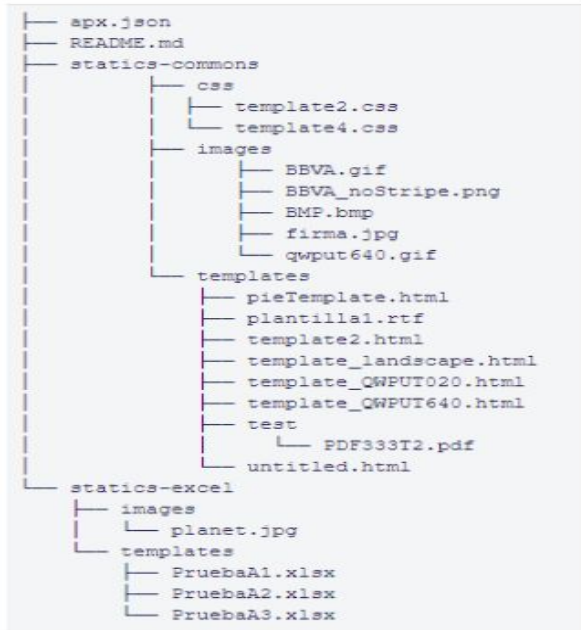
..



 [images](#)

 [templates](#)

- Se permitirá la creación de carpetas adicionales dentro del repositorio y al mismo nivel que la anterior, para dar la posibilidad a los aplicativos de ordenar o estructurar los estáticos funcionalmente en función de sus necesidades. Cada una de estas carpetas deberá nombrarse con el prefijo “statics-” y deberá contener las subcarpetas: *images*, *css*, *templates*. Ejemplo:



- En el despliegue de un recurso de tipo Statics, los ficheros se copiarán en las siguientes rutas destino dentro de la UUAA:
 - Templates: uuaa/templates/
 - Images: uuaa/images/
 - CSS: uuaa/css/

NOTA: En el despliegue, no aplicará la organización funcional de ficheros en carpetas dentro del repositorio comentada en el punto anterior. Es decir, siguiendo con el ejemplo, si tuviéramos dos carpetas: *statics-commons* y *statics-excel*, todos los ficheros contenidos en estas dos carpetas desplegarían en las mismas rutas: *qwpu/templates/*, *qwpu/images/* y *qwpu/css/*.

- No se permiten ficheros con las siguientes extensiones: jar, sh o sql
- No pueden existir ficheros con el mismo nombre dentro del repositorio ya que desplegarán en la misma ruta.

Ciclo de vida de un recurso

Para terminar con la definición de los recursos, vamos a ver algunas claves sobre su ciclo de vida. Más adelante en este documento, revisaremos el modelo de branching que vamos a seguir y entraremos en detalle con cómo va a evolucionar la versión de un recurso y todos sus componentes APX. Para empezar, vamos a ver lo fundamental para nuestro recurso:

Todos nuestros componentes APX, estén en el recurso que estén, tienen una versión. Y esa versión es del tipo [major].[minor].[patch]-[RELEASE|SNAPSHOT], prescindiendo del sufijo cuando sea una versión final.

Dentro de un recurso todos sus componentes tienen la misma versión y evolucionan a la par. También en las agrupaciones como la Unidad de Despliegue.

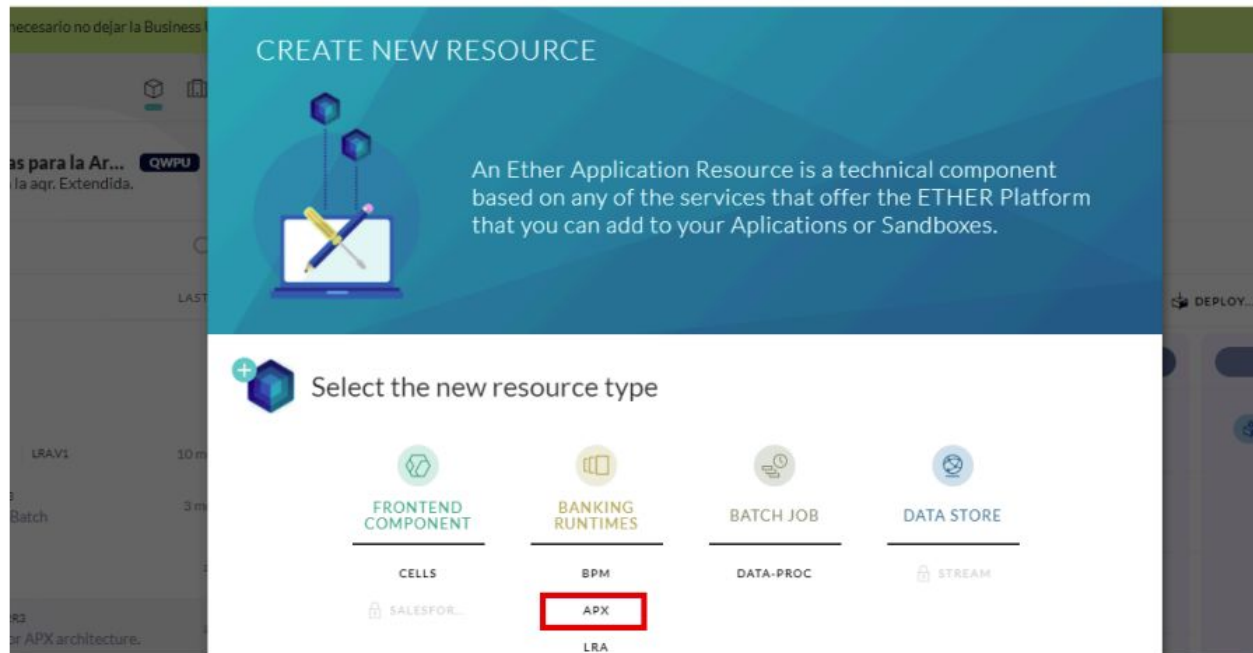
Todos los componentes APX que forman un recurso se despliegan a la vez.

Vamos a verlo con un ejemplo. Tengo un recurso tipo librería. El parent tiene la versión 1.2.0, al igual que la interfaz y la implementación. Encuentro un error en mi implementación y hago un hotfix (más info de qué se considera un hotfix en la sección de modelo de branching), lo cual sube la versión de todo el recurso a la 1.2.1. Redespliego mis componentes, interfaz e implementación, con esta nueva versión para que se corrija el error.

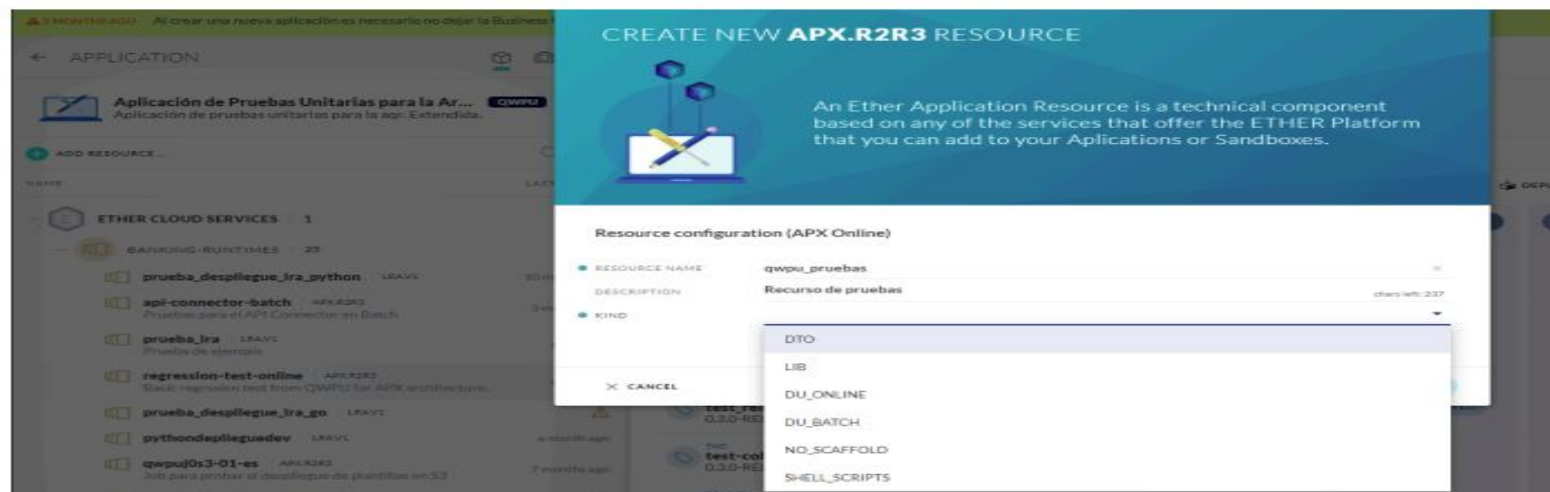
En resumen, cuando pensamos en un recurso tenemos que verlo como una unidad, con todo lo que implica, aunque pueda tener más de un componente APX.

Creación de un recurso

Para crear un recurso de APX tenemos que navegar hasta nuestra aplicación y clicar en 'add resource'.



En el asistente seleccionamos 'APX' y en la siguiente pantalla nos pedirá los datos de nuestro recurso. Una vez indicado el nombre y la descripción de nuestro recurso tenemos que seleccionar el 'kind' o tipo de recurso:



En el asistente podremos observar que en caso de librerías y DTOs nos aparecen campos adicionales. Estos campos son 'code' (en ambos casos) y 'domain' (solo en el caso del DTO):

- **Code:** Código de tres números o letras mayúsculas. Define el código que va a tener nuestro componente APX (por ej. QWPUR**000**). Necesario tanto para librerías como para DTOs.
- **Domain:** el dominio que va a tener nuestro DTO.

Cuando ya tenemos todo listo clicamos en finalizar y empezará la creación del componente. Durante este proceso se van a realizar varias acciones. Para empezar se creará nuestro repositorio de Bitbucket. También se vinculará este repo con Jenkins para poder pasar nuestros pipelines. Una vez hecho esto el TechStack de APX subirá el scaffold de nuestro recurso al repositorio. Nos dejará dos ramas, master, con versión 0.0.0, y develop, con versión 0.1.0-SNAPSHOT. La única diferencia entre estas dos ramas será la versión. Con esto terminamos la creación de nuestro recurso.

Gobierno de componentes APX

Los procesos Globales de Gobierno que se han definido para el alta, la modificación y la modificación express de los recursos APX pueden consultarse en el siguiente [diagrama](#) o [guía de procesos](#).

El detalle de cada uno de estos procesos y su navegación en JIRA Global, se encuentra en los siguientes documentos:

- [Detalle del Proceso de Alta de Componentes APX.](#)
- [Detalle del Proceso de Modificación de Componentes APX.](#)
- [Detalle del Proceso de Modificación Express de Componentes APX.](#)

A continuación se facilitan los documentos de soporte a los procesos con foco en el rol de Developer:

- [Alta Servicio APX \(visión Developer\)](#)
- [Modificación Servicio APX \(visión Developer\)](#)
- [Modificación Express Servicio APX \(visión Developer\)](#)

NOTA: los recursos de tipo Shell Script no pasan por los procesos de Gobierno.

Modelo de ramas

A partir del **10/08/2022** se implantará en APX un nuevo [modelo de ramas unificado para Arquitecturas](#) que aplicará a cualquier tipo de recurso APX (UD, Librería, DTO, Shell Script y Estáticos), quedando deprecados los actuales: [modelo de APX](#) y [modelo para recursos de tipo Shell Script y Estáticos](#). Este nuevo modelo, también basado en gitflow y con soporte a múltiples releases, presenta algunos cambios respecto a los anteriores que es importante tener en cuenta:

- Se elimina la rama **master** pasando a ser **develop** la rama origen del código
- Para solucionar bugs sobre una rama release se usarán ramas **bugfix** (si la release que queremos arreglar no está desplegada en Producción) o ramas **hotfix** (si la release que queremos arreglar está desplegada en Producción)
- La creación de una rama release o el merge de un hotfix sobre la rama release desencadenará automáticamente la creación de una **release version en Consola Ether**
- Para recursos de tipo [Shell Script](#) o [Estáticos](#) aplicará el mismo modelo que para el resto de recursos y por tanto, será necesario trabajar con ramas **release, bugfix y hotfix**.

Los proyectos que ya estuvieran trabajando en sus repositorios con el modelo de ramas deprecado no sufrirán gran impacto, sólo deberán tener en cuenta las siguientes consideraciones:

- **La rama master quedará discontinuada**, a efectos prácticos no genera ningún impacto, sólo dejará de actualizarse con futuros cambios.
- La rama RC se mantiene deprecada y no tiene cabida en el nuevo modelo. Se recomienda que las ramas RC a día de hoy abiertas y en desuso sean cerradas.
- Para recursos de tipo **Shell Script y Estáticos**:
 - No se podrán integrar nuevos cambios en la rama master ni realizar despliegues desde ésta.
 - Para desplegar cualquier cambio en Producción, será necesario abrir una rama release desde develop desplegar desde ésta.

Para los proyectos cuyos repositorios fueron migrados a la Developer Experience con el antiguo modelo de ramas, que aún no hayan realizado ningún despliegue en Producción desde Consola Ether y que necesiten implementar algún cambio, se recomienda seguir los siguientes pasos:

- Primero, **abrir una rama release desde develop** antes de implementar ningún cambio. De esta forma, mantendremos una rama release con el código que tenemos en Producción intacto.
- Segundo:
 - Si el cambio que queremos implementar es una nueva funcionalidad:
 - Abriremos una rama feature desde develop y trabajaremos con normalidad siguiendo el nuevo modelo de ramas

La automatización del modelo

APX proporciona algunos procesos de automatización a través de Jenkins que aplican principalmente en la creación de la rama release, modificación de versiones y propagación de cambios. En adelante iremos comentando cuáles de los procesos están automatizados y qué papel tiene el desarrollador en ellos.

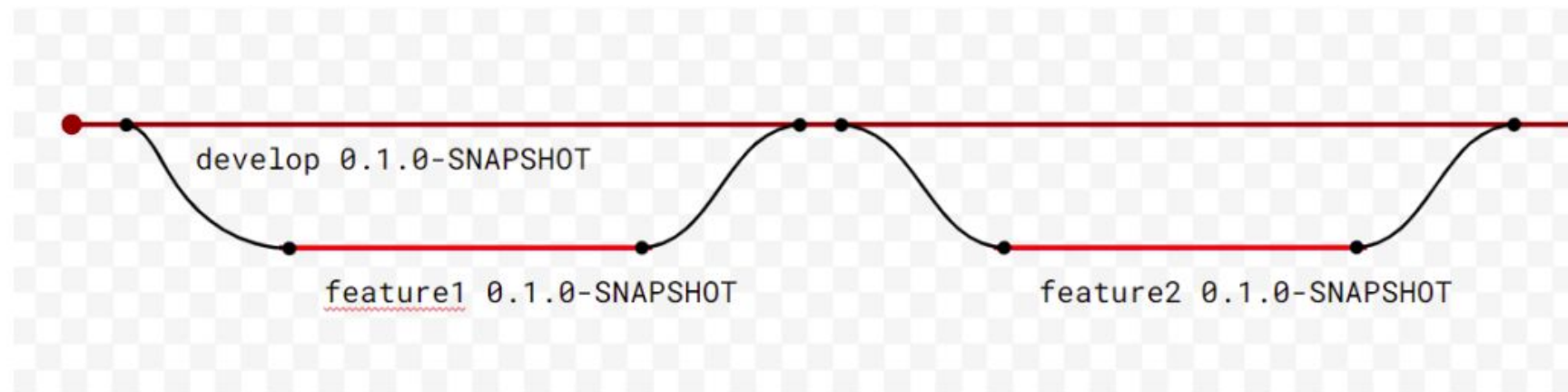
Nombrado de las versiones

El versionado semántico que se establece dentro del modelo de ramas de APX es 'X.Y.Z', añadiendo el sufijo '-[SNAPSHOT]' si se trata de una versión provisional.

- Major version X ($X.y.z \mid X > 0$) Se incrementará cuando se produzcan cambios incompatibles con la versión anterior. Patch y minor son reiniciadas a 0 cuando una versión mayor es incrementada. Ejemplo: 2.0.0
- Minor version Y ($x.Y.z \mid Y > 0$) Se incrementará cuando se introduzca funcionalidad nueva y compatible con la versión anterior, puede incluir cambios a nivel de parches. Patch es reiniciada a 0 cuando una versión menor se incrementa. Ejemplo: 1.3.0
- Patch version Z ($x.y.Z \mid Z > 0$) Se incrementará cuando se introduzcan correcciones de errores (un cambio interno que corrige un comportamiento incorrecto). Ejemplo: 1.3.1

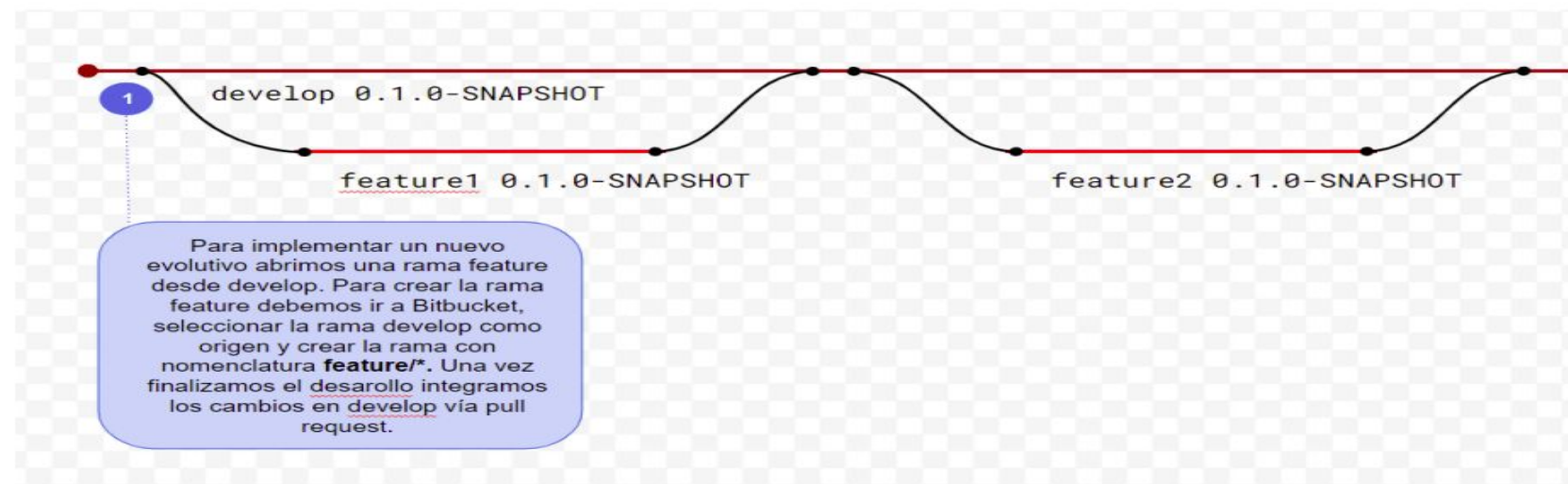
Rama develop

La rama **develop** es el origen del código de nuestro repositorio y rama principal de desarrollo. En esta rama es donde se aplica el scaffold de APX al crear un nuevo recurso.



Nomenclatura	'develop'
Origen	N/A, es la rama origen de nuestro código
Integración vía pull request	N/A
Versionado de artefactos	Los artefactos generados de esta rama tienen versión ' -SNAPSHOT ' y son subidos a Artifactory
Tipo de rama	Permanente
Creación	N/A, se crea automáticamente con la creación de nuestro recurso
Despliegue	Los artefactos pueden ser desplegados en entornos DE o EI

Las ramas **feature** son las ramas donde se implementarán los evolutivos de nuestra versión



Nomenclatura	'feature/*'
Origen	develop
Integración vía pull request	Sí
Versionado de artefactos	Los artefactos generados a partir de esta rama tienen versión '-SNAPSHOT' y NO son subidos a Artifactory . Es decir, no genera dependencias que puedan ser consumidas desde otros recursos (puede consultarse más información sobre cómo trabajar con ramas feature en la nota al final de este apartado)
Tipo de rama	Efímera (se borra después de mergear)

Creación	La creación de una nueva rama feature puede realizarse bien desde Bitbucket o bien construyendo desde el job Jenkins de la rama develop, 'build with parameters' y rellenando el parámetro 'generateFeature' con el nombre de la rama
Despliegue	Los artefactos pueden ser desplegados en entornos DE o EI

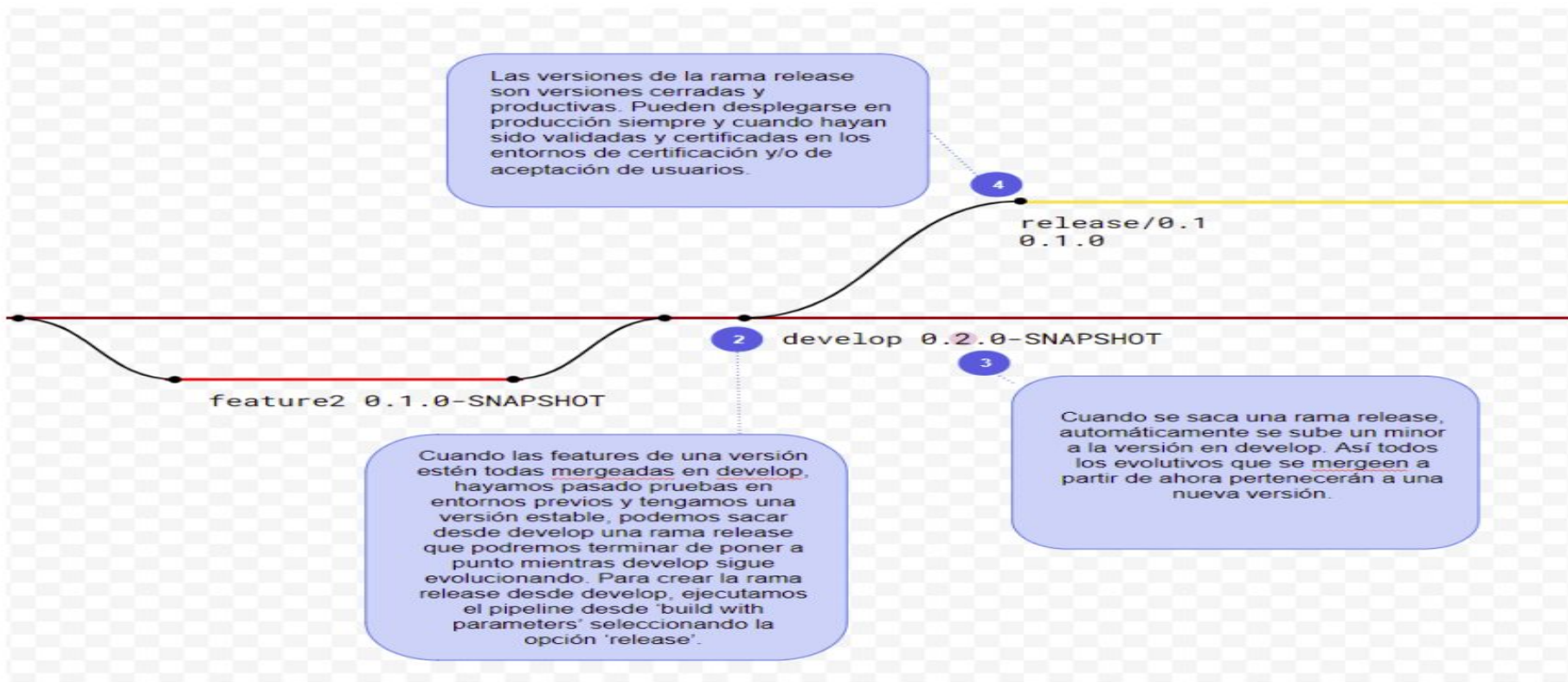
NOTA ¿Cómo trabajar con ramas features?

A la hora de desarrollar con ramas features, hay que tener en cuenta que estas ramas, en tiempo de compilación, no suben artefactos a Artifactory, la rama develop, por el contrario, sí.

Por este motivo, si se necesita implementar dos componentes APX que dependan entre sí, por ejemplo una librería y su DTO, es necesario implementarlos en ramas feature independientes y mergear a develop para disponibilizar las dependencias en Artifactory. En el ejemplo de la librería y DTO, habría que proceder de la siguiente forma:

1. Implementamos el DTO en la rama *feature1*, compilamos a través del pipeline y mergeamos *feature1* a develop. En este momento el DTO ya se encuentra disponible en Artifactory para ser utilizado como dependencia.
2. Implementamos la librería en la rama *feature2* haciendo uso de la dependencia de nuestro DTO, compilamos a través del pipeline y mergeamos *feature2* a develop. En este momento nuestra librería ya se encuentra disponible en Artifactory para ser utilizada como dependencia.

Las ramas **release** contienen versiones finales de software. Pueden ser versiones ya desplegadas en producción o versiones desplegadas en entornos pre-productivos que necesitamos terminar de poner a punto pasando pruebas de certificación por ejemplo.



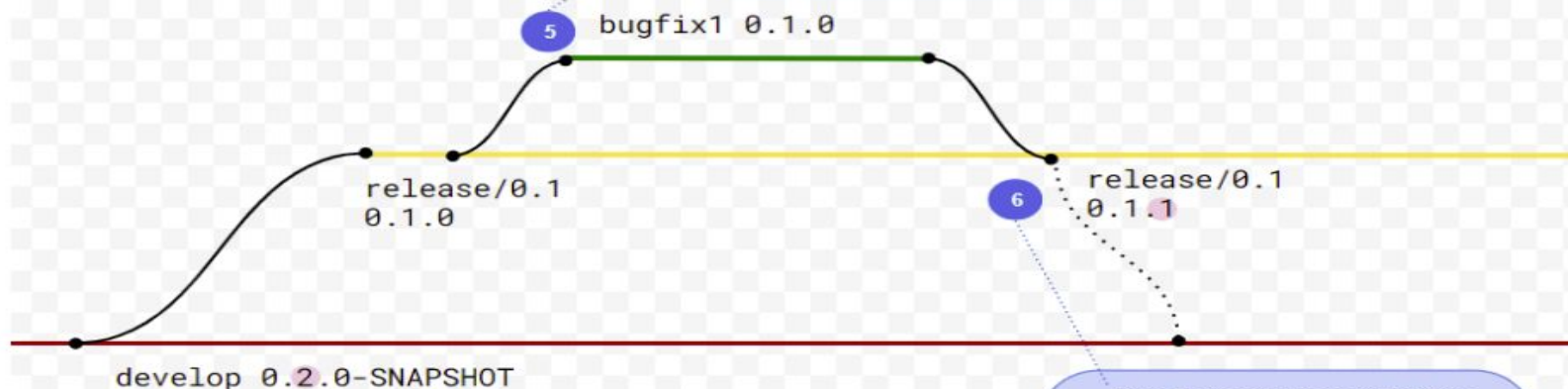
Nomenclatura	'release/x.y''
Origen	develop
Integración vía pull request	N/A
Versionado de artefactos	Los artefactos generados a partir de esta rama tienen versión final y son subidos a Artifactory, donde son únicos y no pueden ser reemplazados.
Tipo de rama	Semi-permanent. The development team decides when a release is no longer needed (because newer releases have replaced it). In case it is no longer needed, it can be deleted through Bitbucket.
Creación	Para crear una nueva rama release desde develop, hay que ejecutar el job de Jenkins , 'build with parameters' y seleccionar la opción 'release'
Despliegue	Los artefactos pueden ser desplegados en todos los entornos incluido PR

Rama bugfix

Es la rama donde se solucionan los bugs que afectan a una release que aún no ha sido desplegada en producción. La integración de un bugfix en la rama release desencadena la propagación automática* del cambio hacia develop y releases superiores.

NOTA*: en el apartado [Propagaciones automáticas y resolución de conflictos](#) se explica el detalle del proceso y cómo actuar en caso de producirse conflictos

Si detectamos un bug en una release en un entorno preproductivo (certificación o aceptación de usuarios) abriremos un **bugfix** para solucionarlo. Para crear la rama **bugfix** debemos ir a Bitbucket, seleccionar la rama release origen y crear la rama con nomenclatura **bugfix/***



Cuando mergeamos un bugfix o hotfix en una rama release, se aumenta automáticamente el patch de la versión de la rama release y se realiza una propagación automática hacia develop. Además, si hay ramas release con versiones superiores, el cambio también se propagará hacia ellas.

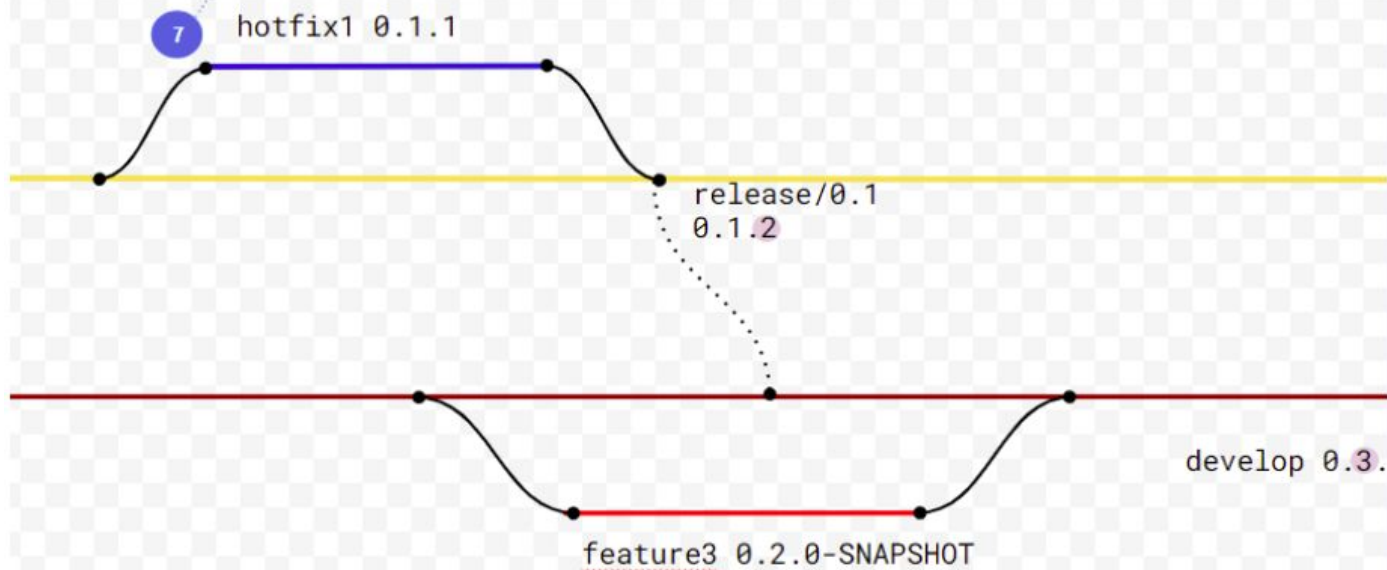
Nomenclatura	'bugfix/*'
Origen	release
Integración vía pull request	Sí
Versionado de artefactos	Los artefactos generados a partir de esta rama tienen versión final y NO son disponibilizados en Artifactory para su dependencia desde otros recursos.
Tipo de rama	Efímera (se borra después de mergear)
Creación	La creación de una nueva rama bugfix puede realizarse bien desde Bitbucket o bien construyendo desde el job Jenkins de la rama release, 'build with parameters' y rellenando el parámetro 'generateBugfix' con el nombre de la rama.
Despliegue	Los artefactos pueden ser desplegados en entornos DE o EI

Rama hotfix

Es la rama donde se solucionan los bugs que afectan a una release desplegada en producción y necesitan arreglarse de forma urgente. La integración de un hotfix en la rama release desencadena la propagación automática* del cambio hacia develop y releases superiores.

NOTA*: en el apartado [Propagaciones automáticas y resolución de conflictos](#) se explica el detalle del proceso y cómo actuar en caso de producirse conflictos

Si detectamos un bug en una release que ya tenemos desplegada en Producción abriremos un hotfix para solucionarlo. Para crear la rama hotfix debemos ir a Bitbucket, seleccionar la rama release origen y crear la rama con nomenclatura **hotfix/***



Nomenclatura	'hotfix/'
Origen	release
Integración vía pull request	Sí
Versionado de artefactos	Los artefactos generados a partir de esta rama tienen versión final y NO son disponibilizados en Artifactory para su dependencia desde otros recursos.
Tipo de rama	Efímera (se borra después de mergear)
Creación	La creación de una nueva rama hotfix puede realizarse bien desde Bitbucket o bien construyendo desde el job Jenkins de la rama release, 'build with parameters' y rellenando el parámetro 'generateHotfix' con el nombre de la rama
Despliegue	Los artefactos pueden ser desplegados en entornos DE o EI

A la hora de corregir un bug de una versión en Producción es importante tener en cuenta las siguientes consideraciones:

- Tendrá que abrirse una rama hotfix desde la release, implementar la corrección y mergear vía pull request sobre la misma release. En estos casos, es fundamental identificar en qué release se introdujo el bug. Veámoslo con un ejemplo:
 - Tenemos tres ramas release: release/1.0, release/1.1 y release/1.2
 - Detectamos un bug en la versión 1.2.
 - Tras analizar el código identificamos que el bug se introdujo en la versión 1.1
 - Sacamos un hotfix desde la release/1.1, corregimos el código, y hacemos pull request sobre esa misma release.
 - Al mergearse, el pipeline automáticamente propagará el cambio a develop (para no sacar más versiones con ese fallo) y a las releases superiores, en este caso release/1.2.

Pull request

El mergeo de cambios entre ramas se realizará siempre desde Bitbucket vía pull request. Existirán dos tipos de pull requests:

- Pull request manuales: son las que tienen abrir los desarrolladores para solicitar la integración de los cambios implementados:

Rama origen	Rama destino
feature	develop
bugfix	release
hotfix	release

Desde Bitbucket, iríamos a *Create pull request*:

Bitbucket Your work Projects Repositories People GRaaS

Create pull request

qwpu-stat... develop → qwpu-stat... master [Change](#)

Title* Develop

Description

Reviewers a.garcia s.fernandez

[Create](#) [Cancel](#)

En el campo reviewers el desarrollador podrá introducir los usuarios que desee para revisar el pull request. Se requiere mínimo un approve para poder mergear

Pull request automáticas que son las que se gestionan de forma transparente para los usuarios en la propagación de cambios que realiza el pipeline automáticamente. En estas pull request también están configurados por defecto como reviewers los usuarios con rol Release Manager. En este caso, estos usuarios podrán revisar los cambios pero el merge lo hará automáticamente el pipeline en ausencia de conflictos. Más información en el siguiente apartado [Propagaciones automáticas y resolución de conflictos](#).

Cuando se mergea un pull request desde un bugfix o hotfix a una rama release, este merge se propaga automáticamente a la rama develop y ramas releases superiores (versiones superiores a la release sobre la que se ha hecho el hotfix)

Se pueden producir 2 escenarios al hacer estas propagaciones automáticas.

Escenario 1: Propagación automática, no requiere atención del developer.

En este caso, la fusión de una rama en otra es limpia y no se requiere ninguna acción manual.

Veamos un ejemplo, se ha mergeado un hotfix sobre la release/0.4, esto además del propio merge del hotfix, lanzará la propagación de este código a la rama develop (en este ejemplo al no existir una release superior a la 0.4, no se propaga nada más) y enviará por email las notificaciones de Bitbucket

de la propagación a los default reviewers establecidos en el repositorio. Por defecto, los default reviewers son los usuarios con rol Release Manager (incluidos Release Manager Global y Release Manager Local en el caso de UAs multicountry)

Como se ve en la siguiente imagen, el proceso ha ido bien por lo que no hace falta hacer nada más.

CloudBees CI Managed Master

Jenkins > APX > BBVA_GL_Apps_APX > gl.qwpw.app-id-8700.dsg > ud-online-regresion > release/0.4

Branch release/0.4

Full project name: gl.qwpw.app-id-8700.dsg/ud-online-regresion/release%2F0.4

[Recent Changes](#)

Stage View

Average stage times:
(Average full run time: ~2min
16s)

	Checkout Global Library	Checkout SCM	Update Patch	Propagate merge to develop
Average	25s	689ms	41s	34s
#15 Jun 25 13:04 No Changes				
#14 Jun 25 13:02 1 commit	25s	652ms	41s	34s
#13 Jun 25 11:00 No Changes	26s	726ms		

Historia de tareas [Tendencia](#)

find

- #15 25-Jun-2021 11:04
- #14 25-Jun-2021 11:02 PR Merged
- #13 25-Jun-2021 9:00

Además en la vista de tareas de Jenkins aparecerá un pipeline creado para realizar esta propagación, aparece como eliminado ya que una vez efectuada la fusión se borra automáticamente. Con el paso del tiempo desaparecerán también de esta vista

Jenkins > APX > BBVA_GL_Apps_APX > gl.qwpu.app-id-8700.dsg > ud-online-regresion >



Up



Status



Configure



Scan Multibranch Pipeline Now



Scan Multibranch Pipeline Log



Multibranch Pipeline Events



Delete Multibranch Pipeline



Personas



Historial de trabajos



Relacion entre proyectos



Comprobar firma de archivos



Groups



Roles



Move/Copy/Promote



Support



Open Blue Ocean



Bitbucket



Rename



Config Files



Pipeline Syntax



Credentials



ud-online-regresion



¡Novedades Octubre 2020!

- Jenkins 2.249.2.3 disponible. [Info aquí](#)

auto-generated by stacker

Disable Multibranch Pipeline

Branches (9)

Pull requests (0)

Tags (21)

S	W	Name ↴	Último Éxito	Último Fallo	Última Duración	Fav
		develop	23 Hor - #13	N/D	9 Min 25 Seg	
		feature/lorueba	3 días 19 Hor - #2	N/D	7 Min 25 Seg	
		master	23 Hor - #6	N/D	4 Min 37 Seg	
		mergeapx/merge-release-0.4-into-develop-34	N/D	N/D	N/D	
		mergeapx/merge-release-0.4-into-master-34	N/D	N/D	N/D	
		release/0.1	N/D	N/D	N/D	
		release/0.2	2 días 2 Hor - #1	N/D	10 Min	
		release/0.3	7 días 18 Hor - #2	N/D	7 Min 47 Seg	
		release/0.4	5 Min 5 Seg - #14	N/D	2 Min 58 Seg	

Icono: [S](#) [M](#) [L](#)[Guía de iconos](#) [Atom feed para todos](#) [Atom feed para fallas](#) [Atom feed para los más recientes](#)

Trabajos en la cola (3)

Escenario 2: Conflictos en la propagación automática, requiere atención del developer.

En este caso, la fusión no ha podido realizarse por encontrarse un conflicto en el código.

Veamos un ejemplo, se ha mergeado un hotfix sobre la release/0.4, pero anteriormente se mergeo una feature sobre la rama develop tocando el mismo código que en el hotfix, esto provocará un conflicto que debe ser resuelto por el developer.

Como en el escenario 1, se lanzará la propagación automática a la rama develop, pero esta vez la propagación hacia la rama develop fallará, quedando dicho stage en estado unstable, lo que se aprecia en la siguiente imagen:

CloudBees
CI

Managed Master

búsqueda

Jenkins > APX > BBVA_GL_Apps_APX > gl.qwpu.app-id-8700.dsg > ud-online-regresion > release/0.4

Up
Status
Changes
Build with Parameters
Ver Configuración
Support
Full Stage View
Open Blue Ocean
Bitbucket
Samuel
Groups
Roles
Pipeline Syntax

Branch release/0.4

Full project name: gl.qwpu.app-id-8700.dsg/ud-online-regresion/release%2F0.4

[Recent Changes](#)

Stage View

	Checkout Global Library	Checkout SCM	Update Patch	Propagate merge to develop
Average stage times: (Average full run time: ~4min 8s)	25s	536ms	31s	26s
#18 Jun 25 13:58 No Changes				
#17 Jun 25 13:57 1 commit	26s	532ms	31s	26s
#16 Jun 25 13:29 No Changes	24s	541ms		

Historia de tareas Tendencia

find

- #18 25-jun-2021 11:58
- #17 25-jun-2021 11:57
Error propagating merge, auxiliar branches have been created
- #16 25-jun-2021 11:29
Create hotfix
- #15 25-jun-2021 11:04

Enlaces permanentes

- Última ejecución (#18) hace 1 Min 2 Seg
- Última ejecución estable (#16) hace 30 Min

Siguiendo con el ejemplo, en la siguiente imagen vemos la vista de tareas de Jenkins, en ella aparecerá el pipeline creado para realizar la propagación a develop como activo para que el developer pueda trabajar sobre esa rama auxiliar y corregir el conflicto.

Jenkins > APX > BBVA_GL_Apps_APX > gl.qwpw.app-id-8700.dsg > ud-online-regresion >

- Up
- Status
- Configure
- Scan Multibranch Pipeline Now
- Scan Multibranch Pipeline Log
- Multibranch Pipeline Events
- Delete Multibranch Pipeline
- Personas
- Historial de trabajos
- Relacion entre proyectos
- Comprobar firma de archivos
- Groups
- Roles
- Move/Copy/Promote
- Support
- Open Blue Ocean
- Bitbucket
- Rename
- Config Files
- Pipeline Syntax
- Credentials



ud-online-regresion



¡Novedades Octubre 2020!

- Jenkins 2.249.2.3 disponible. [Info aquí](#)

auto-generated by stacker

Desable Multibranch Pipeline

Branches (8)

Pull requests (0)

Tags (22)

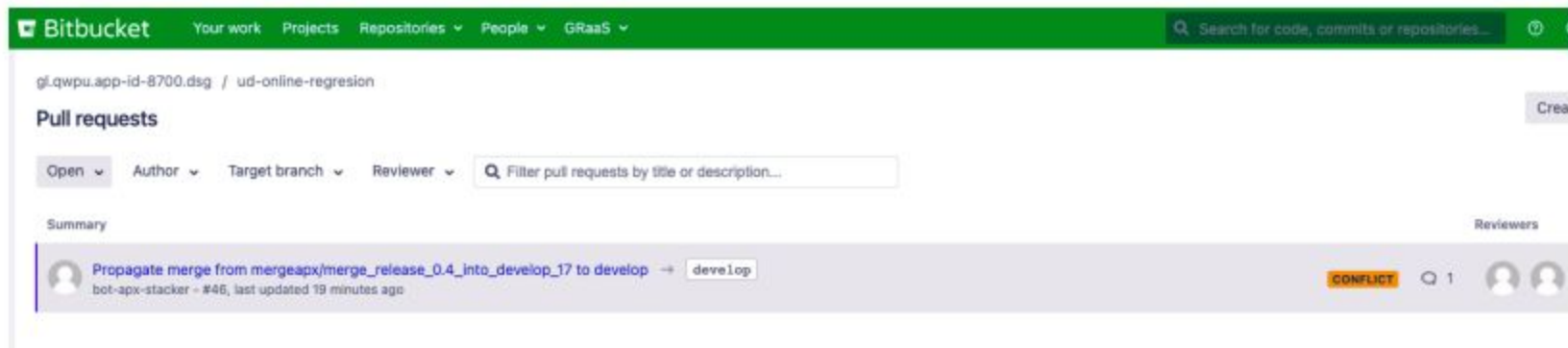
S	W	Name ↓	Último Éxito	Último Fallo	Última Duración	Fav
		develop	20 Min - #15	N/D	8 Min 11 Seg	
		master	7 Min 28 Seg - #28	N/D	5 Min 55 Seg	
		mergeapx/merge_release_0.4_into_develop_17	8 Min 11 Seg - #1	N/D	7 Min 12 Seg	
		mergeapx/merge_release_0.4_into_master_17	N/D	7 Min 46 Seg - #1	2 Min 35 Seg	
		release/0.1	N/D	N/D	N/D	
		release/0.2	2 días 3 Hor - #1	N/D	10 Min	
		release/0.3	7 días 19 Hor - #2	N/D	7 Min 47 Seg	
		release/0.4	9 Min 59 Seg - #17	N/D	2 Min 34 Seg	

Icono:

[Guía de iconos](#) [Atom feed para todos](#) [Atom feed para fallas](#) [Atom feed para los más recientes](#)

Trabajos en la cola

Así mismo en bitbucket aparecerá el pull request de la rama auxiliar creada durante la propagación, en estado CONFLICT y se habrá notificado este pull request vía email de Bitbucket a los default reviewers establecidos en el repositorio:



A partir de aquí este pull request debe tratarse de la misma forma que otros pull request. La no resolución del conflicto y por consiguiente no mergeo de este pull request puede ocasionar importantes errores en las siguientes propagaciones así como divergencias en las ramas.

Controles en ramas

Actualmente, se hacen una serie de validaciones sobre el modelo de ramas, se detallan a continuación:

Nombrado de ramas:

- develop
- feature/*
- release/X.Y
- bugfix/*

- hotfix/*

Rama develop no se pueden borrar porque es permanente

Interacción con develop y release/* solo mediante pull request

Para mergear un **pull request** se requiere:

- Mínimo un approve
- Mínimo una build exitosa
- Ningún "Needs to Work" pendiente de solucionar

Rol Developer:

- desarrolla sobre ramas efímeras feature/* bugfix/* hotfix/*
- realiza pull request sobre develop y release
- puede revisar y mergear pull request sobre ramas develop o release
- despliega recursos en work

Rol Release Manager:

- puede revisar y mergear pull request sobre ramas develop o release
- despliega recursos en live
- puede crear versiones de un recurso desde la Consola

Rol Release Manager Local:

- puede crear versiones de un recurso desde la Consola sobre branch de tipo hotfix (solo aplica para UAs Multicountry) y solo desplegables en su geografía

Nombrado de las versiones:

- Se valida que el nombre de las versiones tenga la nomenclatura correcta: **X.Y.Z**. En ramas develop y feature, las versiones deben contener sufijo '-SNAPSHOT'
- Dentro de un recurso Unidad de Despliegue todos los componentes agrupados deben tener la misma versión

Permisos de despliegue por rol en el modelo de ramas

	DEV	INT	OCT	AUS	PRO
release (RM)	✓	✓	✓	✓	✓
develop (RM)	✓	✓	✗	✗	✗

feature	✓	✓	✗	✗	✗
bugfix	✓	✓	✗	✗	✗
hotfix	✓	✓	✗	✗	✗

✗ - No permitido

✓ - Permitido

✓ - Sólo RM

* - En proceso para ser permitido

Modificación y construcción del recurso

Una vez creado nuestro recurso ya podemos empezar a modificarlo. Para ello, desde la pantalla de información de nuestro recurso podemos acceder al repositorio de Bitbucket que se nos ha creado. Lo clonamos a nuestro local y ya podemos empezar a implementar los cambios en nuestro recurso.

Arquitectura APX provee a los desarrolladores de herramientas para el desarrollo de componentes, como *APX CLI* que nos permitirá crear y añadir componentes de tipo librería, transacción, job o DTO a nuestros recursos o el *Entorno Local de APX* para realizar pruebas de nuestros componentes. Estos son algunos enlaces de interés para desarrolladores de APX que puede consultar en Platform: [Stack to develop with APX](#), [APX CLI](#), [Codelabs](#)

Integración continua

Los cambios que hagamos en el repositorio los iremos registrando en la rama develop mediante commits. Al hacer push de esos commits, automáticamente se disparará una ejecución del pipeline de Jenkins de la rama que acabamos de pushear.

El pipeline de Jenkins, a través de sus distintos steps, se encargará de realizar tareas de integración continua como construir el recurso, ejecutar tests unitarios, ejecutar controles de calidad y seguridad, etc, así como registrar y disponibilizar el recurso para su futuro despliegue.

Branch develop



Stage View

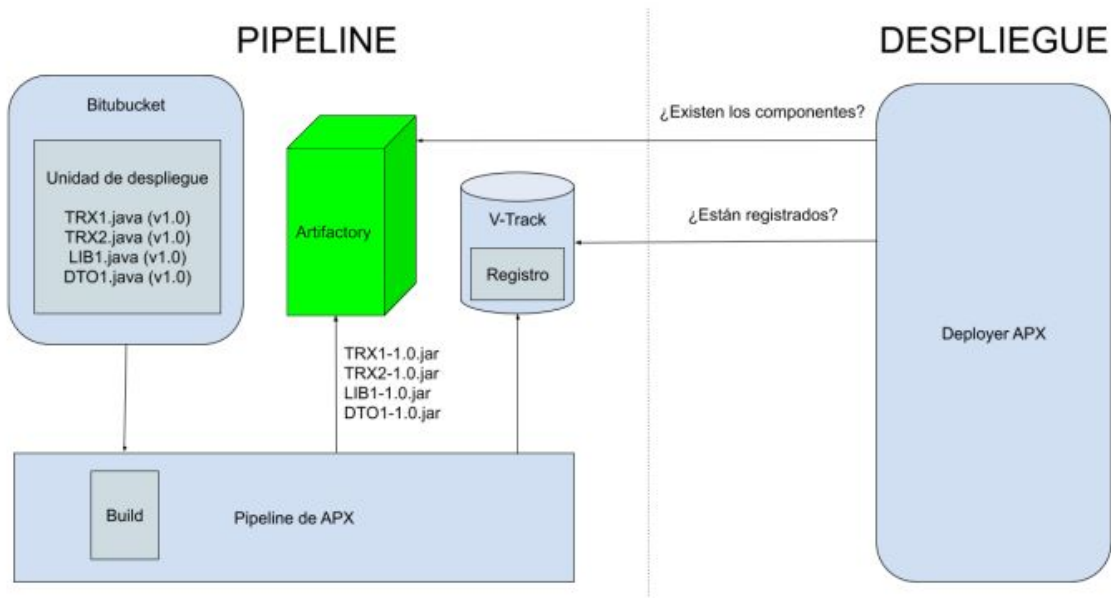


Veamos algunos de los principales steps con más detalle:

- APX Prerequisites: Ejecuta enforcements a través de Samuel que validan distintos aspectos como:
 - La versión de Arquitectura APX utilizada es una de las tres últimas.
 - Los componentes dentro de un recurso comparten la misma versión.
 - Cumplimiento de la convención establecida respecto al naming en el modelo de ramas.
 - Detección de vulnerabilidades en código a través de Chimera.
 - Etc... Puede consultar en [Samuel](#) el catálogo completo de comportamientos APX.
- Build: Compila el código fuente de nuestro recurso.
- Test: Ejecuta los tests unitarios. Puede consultar para más información en [Testing en APX](#).
- Sonar: Realiza el control de calidad a través de SonarQube. Enlaces de interés: [Quality Gate](#) definido para APX y listado de [reglas Sonar APX](#) que se ejecutan para asegurar que se cumple con las buenas prácticas definidas por la Arquitectura.
- Deploy Artifactory: Subida de los artefactos al repositorio de binarios. Solo para ramas develop y release (Las ramas efímeras feature, bugfix y hotfix solo suben artefactos al repositorio de efímeros y no pueden ser usados como dependencias)
- Tracing in vTrack: Registro de la build en vtrack.

El registro en V-Track y Artifactory es especialmente importante porque nos permitirá más a hacer nuestro despliegue. Vamos a ver un poco más en detalle el papel de estas herramientas de despliegue de nuestro recurso.

Lo que tenemos que saber sobre esto es: el pipeline construye los componentes, los deposita en Artifactory y los registra en V-Track, como vemos más abajo:



Visto esto sabemos que para que podamos pasar al despliegue es necesario que se complete el pipeline. Una vez hayamos hecho nuestros cambios, hayamos hecho push y comprobado que todo se pasa correctamente, tendremos el feedback correspondiente en la sección de nuestro recurso en la consola. Si entramos en la pestaña 'audits' podremos ver que dentro de 'builds' está registrada la nuestra, y a partir de ahí podremos seguir con el despliegue.

Testing en APX

Como ya se ha comentado, una de las tareas que realiza nuestro pipeline en el proceso de construcción de un recurso es la ejecución de Test Unitarios. Si en la ejecución alguno de los tests fallara, el pipeline terminaría con error en este punto. Si desea consultar información sobre cómo implementar tests unitarios en APX puede consultar en Platform los siguientes enlaces: [Testing in APX Online](#), [Testing Codelab](#), [Testing in APX Batch](#).

La norma establecida en APX en cuanto a Test Unitarios es tener una cobertura de al menos el 80% sobre el nuevo código. Esto lo valida también el pipeline a través de Sonar, en caso de no llegar a la cobertura exigida, el pipeline terminará con error. (Puede consultar en detalle el Quality Gate definido para APX así como las reglas Sonar aplicadas en APX en los siguientes enlaces: [Quality Gate](#), [reglas Sonar APX](#)).

Para la ejecución de pruebas integradas, la Arquitectura ofrece un Entorno Local APX para el despliegue y pruebas en local de nuestros componentes. Puede descargarlo y consultar más información en Platform: [Entorno Local APX](#), [Local Deploy Codelab](#)

Despliegue de un recurso

Una vez hecha nuestra build y con nuestros componentes registrados en Artifactory es hora de hacer el despliegue. En la Consola, dentro de la pantalla de nuestro recurso, iremos a la pestaña 'release versions'. En esta pestaña vamos a controlar todos los despliegues de nuestro recurso. Los despliegues se organizan en release versions. Una release version es una encapsulación de un tag o una rama que se va a desplegar por los entornos.

The screenshot displays the Artifactory console interface. On the left, the 'APPLICATION' sidebar shows a tree view with 'ETHER CLOUD SERVICES' (1 item) and 'BANKING-RUNTIMES' (22 items). The 'regression-test-online' resource is selected under 'BANKING-RUNTIMES'. The main panel shows the 'RESOURCE' details for 'regression-test-online' (APXR2R3), including a description: 'Basic regrssion test from QWPU for APX architecture.' The 'RELEASE VERSIONS' tab is active, showing a list of versions and a deployment matrix.

VERSION	DEV	INT	PROD
TAG test_release_version 0.3.0-RELEASE	DEPLOYED	DEPLOYED	
TAG test-colombia-console 0.3.0-RELEASE	*	*	
BRANCH release0.3 release/0.3	*	*	

Para crear una release version, hacemos clic en el botón 'Create release version'. En el asistente que se muestra a continuación, deberemos seleccionar el branch o tag que vamos a añadir en la release version e indicar un nombre para la release version. (NOTA: Solo los usuarios con rol Release Manager podrán crear release versions de un recurso. En el caso de UAs multicountry, el rol Release Manager Local podrá crear release versions sobre brach de tipo hotfix y release)

VERSIONING RESOURCE **REGRESSION-TEST-ONLINE**



Release Versions can be created from an specific tag, commit or branch

SOURCE ORIGIN

TAG

SUMMARY

Select the origin code for this new release version

 regression-test-online

 BRANCH

 TAG

 Search tag source...

TAG	LAST COMMIT	BUILD	AUTHOR	LAST UPDATE
 0.1.0-RELEASE	 2ceæb71110...			
 0.2.0-RELEASE	 7d80c6f2808...			
 0.3.0-RELEASE	 a2492d30ff2...		bot-globaldevops	7 months ago

 CANCEL

NEXT 

VERSIONING RESOURCE

REGRESSION-TEST-ONLINE



Release Versions can be created from an specific tag, commit or branch

SOURCE ORIGIN

TAG

SUMMARY

Resource Release Version configuration

RELEASE VERSION NAME

rc-r4

ALIAS

Demo-2020-03-04 (Optional)

TAGS

Tags (Optional)

CANCEL

PREV

CREATE RELEASE VERSION

Una vez creada la release version, se nos abrirá una fila con un botón de 'deploy' en cada uno de los entornos.

RESOURCE

APXK2R3

regression-test-online

Basic regrssion test from QWPU for APX architecture.

INFO

RELEASE VERSIONS

AUDITS

POLICIES

CREATE RELEASE VERSION...

DEPLOY...

DATA OBTAINED 20 SECS. AGO

REFRESH IN 10 SECS.

VERSION

DEV

INT

AUS

OCT

PRO

TAG

test_release_version

0.3.0-RELEASE

DEPLOY

DEPLOY

DEPLOY

DEPLOY

DEPLOY REQUEST

Si le damos al botón de 'dev' se nos abrirá un asistente para el despliegue:

DEPLOY RESOURCE VERSION...

 **TransactionRules** | APX
An Ether Application Resource is a technical component based on any of the services that offer the ETHER Platform. [+ more info](#)

DEPLOYMENT CONFIGURATION

1

2

3

4

5

ENVIRONMENT **ESP** **DEV** **INT** **GRE** **AUS** **OCT** **PIL** **PRZ** **PRO**

NS es.hrch.app-id-6349.pro

VERSION TO DEPLOY v2.0 (RC branch)

SCHEDULING DATE 20/07/2020 **WINDOW** 10:00 AM

GOVERNANCE INFO

JIRA TICKET APXGP- 12_ or SOPVII- ticket id...

CRQ CODE Insert

ENGINE PARAMS

DEPLOY MODE: Physics Online

DEPLOYMENT CONFIG PARAMS

DEPL. CONFIG TEMPLATE: **ESP** **PRO** **APX** DealParams Production Online

SAVE NFW

1] Se selecciona (o vendrá preseleccionada la geografía de la aplicación en la que se va a desplegar el recurso.

2] Se selecciona en qué entorno se quiere realizar el despliegue.

3] Se indica cuál de las versiones del recurso, definidas y configuradas previamente, es la que se quiere desplegar.

4] En función del entorno y del tipo de engine, se solicita al usuario que introduzca unos datos u otros. Una política de gobierno, que aplica a APX, es que de cara a tener trazabilidad de los despliegues de un recurso, el Developer debe indicar el *ticket Jira* asociado al recurso APX que se va a desplegar (este ticket es opcional para los recursos de tipo SH/SOL o Estáticos). Este ticket debe encontrarse en el estado correcto

dependiendo del tipo de entorno en el que se va a desplegar.

Disclaimer: Para la geografía México no es requerido ingresar un ticket JIRA, es opcional, pero si se cuenta con uno se puede ingresar aún siendo de tipo MP-.

5] Se selecciona el modo de despliegue o tipo de runtime: Physics Online, Physics Batch, R3 o Dual (este último solo en caso de recursos de tipo librería para poder desplegar simultáneamente en Physics Online y Physics Batch, disponible a partir del 14/12/2022)

6] Como se ha comentado en los puntos anteriores, esta sección ya es opcional y no es necesario seleccionar un Deployment Config.

The screenshot displays the 'DEPLOYMENT CONFIGURATION' window. At the top, there are tabs for 'ENVIRONMENT', 'WORK', and 'LIVE'. Under 'ENVIRONMENT', a dropdown menu is set to 'MEX'. To the right, a row of buttons represents different environments: 'DEV', 'INT' (which is highlighted in blue), 'AUS', 'OCT', and 'PRO'. Below these buttons, a text field shows 'rev.zc35.app-45-282017.int'. Under the 'VERSION TO DEPLOY' section, the text 'pruebaunidad-SNAPSHOT' is displayed. At the bottom, under 'GOVERNANCE INFO', there is a 'JIRA TICKET' field with a dropdown set to 'MP-' and a text input containing '1234'.

Durante el deployment se seguirán una serie de pasos que variarán en función de dónde vayamos a desplegar. Si queremos más información, podemos dar dos clics en la casilla de ese entorno (que ahora mismo estará con un icono de carga y el step por el que vaya del despliegue) y se nos mostrarán enlaces a los logs, y en caso de que hubiera habido un error nos daría el detalle de cual es. En caso de que todo vaya bien, se debería de quedar en 'deployed', indicando que nuestro componente ya está disponible.

DEPLOY RESOURCE REGRESSION-TEST-ONLINE



Resources can be deployed from an specific tag, commit or branch

Resource deploy processing

 DEPLOYING RESOURCE REGRESSION-TEST-ONLINE@TEST_RELEASE_VERSION	IN PROGRESS 
 DEPLOYING RESOURCE	IN PROGRESS 
CHECK_REQUEST	COMPLETED
START_DEPLOY_JOB	IN PROGRESS 
EVALUATE_SAMUEL_ENFORCEMENTS	PENDING --
CONFIG_DEPLOYMENT	PENDING --
GET_ARTIFACTS	PENDING --
SELECT_PROMOTION_SERVICE	PENDING --
SCHEDULED	PENDING --
DEPLOYED	PENDING --

X CLOSE

Despliegue en Producción

Una vez creada la release version y desplegada por todos los entornos previos, el siguiente paso sería el despliegue en Producción. Hay que tener en cuenta, antes de llevar a cabo este despliegue en Producción, que la release version debe haber sido creada a partir de un tag y no desde un commit o branch como se permite en entornos previos para poder realizar pruebas, de lo contrario, no se permitirá el despliegue.

Para desplegar en Producción una release version, haremos 'clic' en el botón 'deploy request' que aparece en el entorno de 'pro' en la fila correspondiente a esa release version y se abrirá un asistente de despliegue muy similar al de entornos previos comentado en el apartado anterior:

VERSION	DEV	INT	AUS	OCT	PRO
 TAG test_release_version 0.3.0 RELEASE	 DEPLOYED	 DEPLOYED	 DEPLOYED	 DEPLOYED	 DEPLOY REQUEST

Al pulsar en “Deploy Request...” aparecerá el siguiente cuadro de diálogo donde se configura la solicitud del despliegue en Producción.

REQUEST DEPLOY FOR RESOURCE REGRESSION-TEST-ONLI...

Resources can be deployed from an specific tag, commit or branch

DEPLOYMENT CONFIGURATION

ENVIRONMENT

GLO

DEV

INT

AUS

OCT

PRO

gl.qwpu.app-id-10662.pro

VERSION TO DEPLOY

0.3.0-RELEASE

DEPLOYMENT TYPE

☒ SCHEDULED ☐ AS SOON AS POSSIBLE

SCHEDULING DATE

Dd-Mm-Yyyy

WINDOW

Select a date to schedule deployment.

JIRA TICKET

APXGP-10

Transacción

APX Component Requ...

Under Constructi...

CRQ

CRQ000001

ENGINE PARAMS

DEPLOY MODE

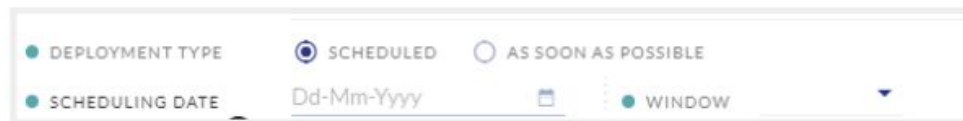
DEPLOYMENT CONFIG PARAMS

TAGS

prueba

Disclaimer: Para la geografía **México** no es requerido ingresar un ticket JIRA, es opcional, pero si se cuenta con uno se puede ingresar aún siendo de tipo **MP**.

Una de las diferencias que presenta el asistente de despliegue de Producción frente al de entornos previos es que ofrece dos tipos de despliegue:



The image shows a configuration panel for deployment. It has two rows. The first row is labeled 'DEPLOYMENT TYPE' and contains two radio buttons: 'SCHEDULED' (which is selected) and 'AS SOON AS POSSIBLE'. The second row is labeled 'SCHEDULING DATE' and contains a text input field with the placeholder 'Dd-Mm-Yyyy', a calendar icon, and a 'WINDOW' dropdown menu.

- 'Scheduled': para realizar despliegues planificados. Deberá seleccionarse la fecha y un horario dentro de las ventanas planificadas. La ejecución de este tipo de despliegues se realizará automáticamente en base a la fecha y ventana indicada y no será necesario que sea ejecutado manualmente por ningún usuario.
- 'As soon as possible': para realizar despliegues bajo demanda. Deberá seleccionarse solamente la fecha de despliegue. NOTA: Al finalizar la solicitud de un despliegue de esta tipología deberá recuperarse el request ID de la petición y posteriormente ejecutar manualmente el despliegue por la herramienta:
 - Cronos, según se indica en la sección "[Ejecución de despliegues en producción por Cronos](#)" al final de este apartado (activo en Colombia, España, Perú y Argentina)
 - Jenkins de LCM, según se indica en la sección "[Ejecución de despliegues en producción por Jenkins LCM](#)" al final de este apartado (activo en México)

Una vez informados datos para el tipo de despliegue, será necesario indicar el código CRQ relacionado a la release version que se va a desplegar y a continuación ya podríamos hacer la solicitud de despliegue.

Desde el punto de vista del Developer, en este momento la solicitud de despliegue que acaba de configurar se habrá enviado al Inbox de Autorizaciones, ya que todo despliegue en Producción requiere de la aprobación por parte de un usuario autorizador (Release Manager/Tech Lead de la UA).

Desde la pestaña de versiones del recurso, podrá ver que se encuentra en estado Pendiente de Autorización.



Autorización de la solicitud de despliegue por parte del Technical Lead/Release Manager

Éste deberá acceder al [Inbox de Autorización de la Consola](#) para autorizar el despliegue. Las solicitudes, tanto “scheduled” como “as soon as possible” no se desplegarán si no están autorizadas.

El Technical Lead puede encontrar información más detallada de la herramienta Inbox en la presentación relativa a su rol dentro de la Development Experience

[\[ETHER UX\] Journeys Ether Development Experience - Rol TechLead](#)

Y en esta otra más específica con la transición de estados desde Consola.

[\[ETHER UX\] Ether Console Inbox Autorización despliegues a Producción](#)

Volviendo al usuario Developer/Release Manager, una vez autorizada su solicitud de despliegue, en función de si el despliegue era de tipo Programado, podrá ver que el despliegue está ya a la espera de que llegue la fecha y ventana(hora) especificadas.

A) Ejemplo de despliegue programado

view: **running versions** ▾

environment: **all** ▾

OCTA

PRO

VE



ACTIVE



SCHEDULED

20/07/2020 | 10:00h

OYED

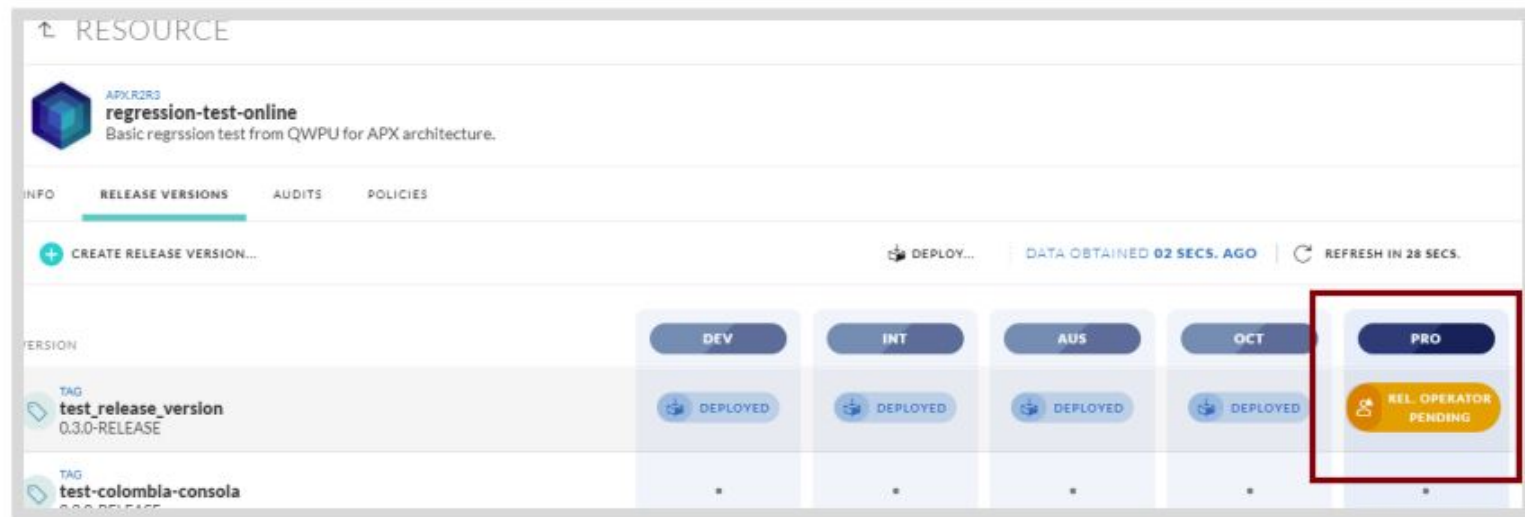


DEPLOYED



DEPLOY...

B) En el caso de un despliegue de tipo 'As soon as possible', nos faltaría por realizar un último paso: recuperar el request ID y ejecutar proactivamente el despliegue en Cronos.



Para obtener el request ID, hacemos clic en el estado de despliegue REL. OPERATOR PENDING y podremos obtener el Request ID de la ventana de detalle del proceso de despliegue.

(NOTA: Existen otras formas de recuperar el Request ID del deployment en Consola, para más detalle consultar el siguiente [link](#)).

Inbox de Autorización (Despliegues de Release Versions en Producción)

El **Inbox de autorizaciones** es una herramienta integrada en Consola accesible desde su menú lateral izquierdo.

Desde aquí se gestionan las solicitudes que envían los Developers a los Release Managers de sus UUA, para que estos puedan autorizar o no el despliegue en producción de una release version de alguno de los recursos técnicos de una Ether Application.

The screenshot displays the 'DEPLOY AUTHORIZATION REQUESTS' interface. On the left, the 'AUTHORIZATION INBOX' section shows 12 requests, with filters for status (All, Pending, Approved, Authorizing, Denied, Error, Cancelled) and dates. The 'PENDING REQUESTS' table lists 14 requests, including columns for Implantation Date, Window, CRQ, Resource, UA, Environment, and Requester. The right panel provides a detailed view of a specific request (TEST 17), showing its UA (MDCO), Tech Stack (APX Online), Environment (ESP), Resource (EOMEJ004_batch_job-master-job), and Implant Date (14/06/2020 - Window 3). It also identifies the UA Authorizer as Ana Gómez López and shows a status history indicating a denied request by Juan Fernández Rodríguez.

Esta herramienta tiene dos puntos de vista y, en función del rol del usuario autenticado, le aparecerán la/s que le aplican a dicho usuario:

Esta herramienta tiene dos puntos de vista y, en función del rol del usuario autenticado, le aparecerán la/s que le aplican a dicho usuario:

- **INBOX (Solicitudes que puedo autorizar si tengo el rol RM, TL)**

Si se navega a esta opción, quiere decir que el usuario ejerce el rol de autorizador respecto a una o varias UUAA y es desde donde podrá ver y Autorizar/Denegar las solicitudes de despliegue que le han llegado (en calidad de Autorizador de la UA a la que pertenezca el recurso que se quiere desplegar en Producción).

- **My Requests (Solicitudes que ha enviado)**

Desde esta otra opción, un usuario con rol Developer/Release Manager puede ver las solicitudes que él/ella ha enviado y así poder consultar en qué estado de Autorización se encuentran.

DOBLE AUTORIZACIÓN

De manera estándar, la autorización para el despliegue en Producción sólo requiere que sea autorizado por alguno de los Release Managers de la UA (o del Technical Lead) pero, en algunas geografías, y para recursos de determinados engines/arquitecturas, la autorización desde el Inbox de Autorizaciones para que pueda ser desplegado un recurso en Producción requerirá que esta deba ser autorizada no sólo por alguno de los responsables de la UA (RM o TL de la geografía de despliegue), sino que también debe tener una segunda autorización por parte de alguna persona del grupo de Gobierno de la geografía de despliegue.

DEPLOY REQUEST



✓

AUTHORIZE

✗

DENNY...

CRQ CODE:

R5ABCD

REQUESTER:

carlos.lopez@bbva.com

REQUEST ID

TEST 17

UA:

MDCO

TECH STACK:

APX Online

ENVIRONMENT:

MEX

PRO

RESOURCE:

...EOMEJ004_batch_job-master-job

IMPLANT. DATE:

14/06/2020 - Window 3

UA AUTHORIZER/S

Pending

 Project Manager
Ana Gómez López

 Maintenance Manager
Juan Fernández Rodríguez

MEX APX COMMITTEE AUTHORIZER/S

Authorized

 Mex Committee Member
Ana Gómez López

 Mex Committee Member
Juan Fernández Rodríguez

 Mex Committee Member
Mario Fernández Gómez

HISTORY

 Authorized by Committee [1/2] 15:32 - Today
Mario Fernández Gómez

 Registered Authorization... 8:24 - Today
BOT KHARON

 Send Authorization 8:23 - Today
Juan Fernández Rodríguez

 Sended Deploy Request 10:36 - Yesterday
Carlos López

Este requisito de doble autorización, de cara a recursos APX, está activado para los despliegues en México (desde Q4'2021)

Las personas de este grupo de Gobierno local tendrá unos o varios administradores que se encargarán de gestionar qué personas quieren incluir en él, y los miembros de este grupo deberán entrar en el Inbox de Autorizaciones de la Consola.

En el panel de detalle de la solicitud seleccionada, en los casos donde esté activada la doble autorización, aparecerán dos grupos de autorizadores. En un primer grupo aparecen los de la geografía de la UA (Tech Lead y/o Release Managers locales) y en un segundo grupo las personas que formen parte del grupo autorizador (gobierno de la geografía).

En estos casos, el estado global de la Solicitud de Autorización aparecerá como **Pendiente** mientras falte alguno de los dos checks por validar.

Una vez que la solicitud está autorizada, es cuando técnicamente comienza el proceso de despliegue de la manera usual y la petición queda registrada en *Kharon*.

Consideraciones a tener en cuenta

De cara a las solicitudes de despliegue en Producción y relacionado con el modo en que se pueden autorizar las solicitudes, se deben considerar los dos siguientes aspectos:

1. Esta solución de la doble autorización actualmente disponible no es una solución estratégica ya que en un futuro, cuando estén integradas piezas como el Remedy y/o JIRA con la Consola, se desea que el CRQ asociado a una solicitud de despliegue sea el mecanismo por el cual la Consola Ether automáticamente conozca si el despliegue está autorizado o no por parte de ese comité o panel de gestión del cambio de una geografía.
2. Por causas técnicas, a día de hoy la Consola Ether actualmente no puede enviar notificaciones vía correo electrónico a los responsables de autorizar un despliegue, por lo que el Developer (o el Release Manager de la UA) deberá comunicar manualmente a dichos responsables la necesidad de que deban entrar en el inbox para autorizar solicitudes pendientes que tengan. En el panel de detalle aparecerá una sección con los nombres de los miembros del grupo de Autorizadores de gobierno de su geografía.

Para entrar en el Inbox de Autorizaciones se puede hacer desde el siguiente link:

<https://console.platform.bbva.com/inbox/list>

El día de la implantación se procederá al despliegue de forma manual y hasta entonces el despliegue reflejará que está en estado *Requested*.

Para ver información más detallada sobre el Inbox de Autorización se puede ir a la sección del Inbox de Autorización del journey de la TechU del Overview de la Development Experience.

 **[ETHER UX] Development Experience**

Ejecución de despliegues en Producción por Jenkins LCM

Los despliegues productivos de tipo “As soon as Possible”, también conocidos como “manuales” o “on-demand”, se deberán ejecutar hasta final de Enero para España según se indica en el apartado “implantaciones Correctivas” de la [Guía del Desarrollador LCM para APX de España](#).

Para Colombia, seguir el apartado siguiente de despliegues por Cronos.

Ejecución de despliegues en Producción por Cronos



NOTA: El proceso de despliegue por Cronos descrito en este apartado se aplicará para España a partir de Enero de 2021. Los jobs de Cronos de esta sección estarán deshabilitados hasta dicho momento.

La herramienta que se va a utilizar para la ejecución de los despliegues productivos de APX es Cronos, herramienta a la que se accede a través de la URL <https://bbva-cronos.appspot.com/>. Cronos es un servicio Ether que sirve para la orquestación, planificación y ejecución de tareas. Para soportar la operación de despliegue se utilizará el elemento "Job" existente en Cronos.

- El job de Despliegue estará disponible en los siguientes namespaces y regiones en función de la geografía de despliegue:
 - España (ESP) → NS: es.qe56.apx-app-deployments Región: live-01
 - Global (GLO) → NS: gl.qe56.apx-app-deployments Región: live-01
 - Colombia (COL) → NS: co.qe56.apx-app-deployments Región: live-02
 - México (MEX) → NS: mx.qe56.apx-app-deployments Región: live-02
 - Perú (PER) → NS: pe.qe56.apx-app-deployments Región: live-02

Actualmente, los jobs de Despliegue estarán disponibles para los Release managers de las Aplicaciones con recursos APX, únicamente para las geografías de ESP y GLO. Para el resto de geografías, esta labor queda restringida y debe ser solicitada al rol centralizado para cada una de esas geografías.

Ejemplo para el caso de España, accediendo a través de la siguiente URL

<https://bbva-cronos.appspot.com/#/live-01/ns/es.qe56.apx-app-deployments/jobs/>.

El Namespace y Región se puede seleccionar en Cronos en los desplegables que aparecen en la esquina superior derecha.

Actualmente, los jobs de Despliegue estarán disponibles para los Release managers de las Aplicaciones con recursos APX, únicamente para las geografías de ESP y GLO. Para el resto de geografías, esta labor queda restringida y debe ser solicitada al rol centralizado para cada una de esas geografías.

Ejemplo para el caso de España, accediendo a través de la siguiente URL

<https://bbva-cronos.appspot.com/#/live-01/ns/es.qe56.apx-app-deployments/jobs/>.

El Namespace y Región se puede seleccionar en Cronos en los desplegables que aparecen en la esquina superior derecha.

The screenshot displays the Cronos application interface in a web browser. The address bar shows the URL: `bbva-cronos.appspot.com/#/live-01/ns/es.qe56.apx-app-deployments/jobs/`. The interface includes a sidebar on the left with navigation options: Plan, Dashboard, Jobs, Journal, Cronos Services, and Namespace. The main content area features a header with filters for Namespace (set to `es.qe56.apx-app-deploym`) and Region (set to `live-01`). Below this, there are four summary cards: Total Jobs (1), Job More Effective (None), Job Most Unstable (None), and Job More Executed (None). A 'New Job' button and search filters are also present. The 'JOB LIST' section contains a table with one entry.

Name	NS	Version	Schedule	Timezone	Last Execution	Last Status	Job State
deploy-es-apx-app-by-request-id	es.qe56.apx-app-deployments	6	None	UTC	-	-	OFF

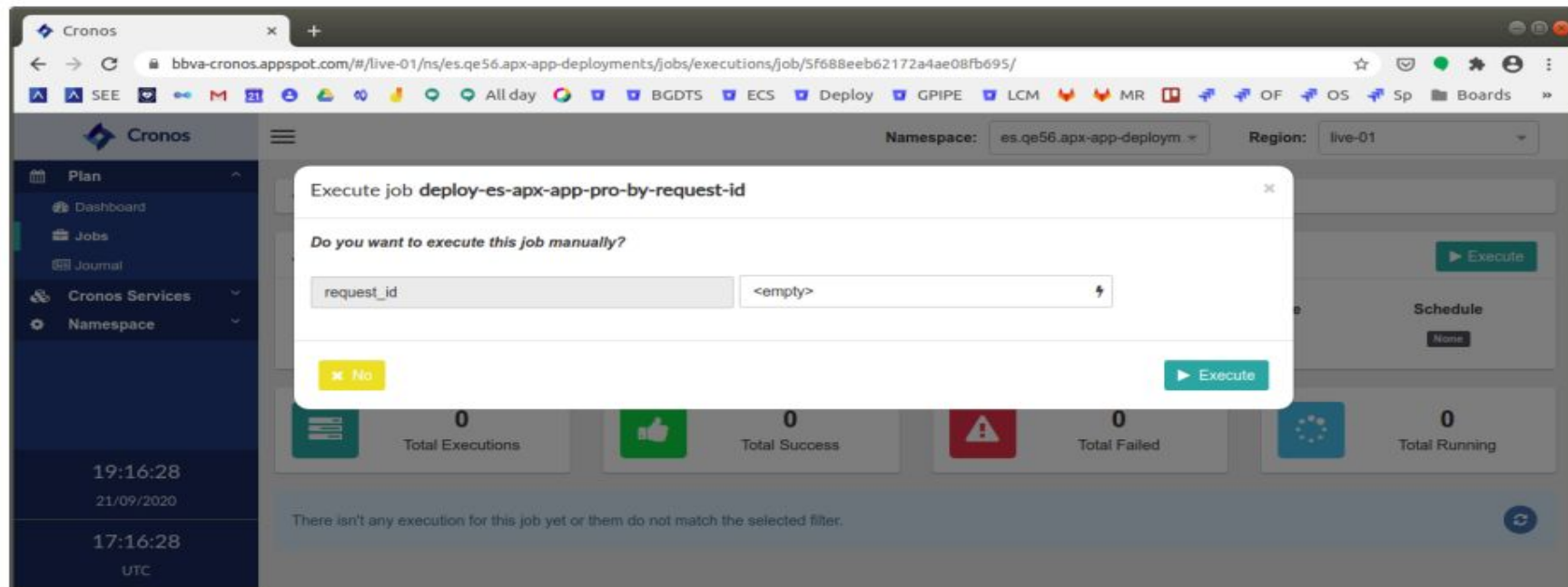
Showing 1 to 1 of 1 entries

Ejecución de Despliegue para Peticiones “as soon as Possible”

Utilizando el job [deploy-es-apx-app-pro-by-request-id](#), especificando como parámetro uno o varios request IDs obtenidos de la solicitud de despliegue.

JOB #1 OFF		Edit Job		Execute
Job Name	Current version	Start Date	End Date	Schedule
deploy-es-apx-app-pro-by-request-id	0	UTC 21/09/2020 11:28:00	-	None

- En caso de quererse desplegar más de uno, los requests IDs se especificarán como una cadena con cada uno de ellos separados por comas y sin espacios entre ellos. Ejemplo: KH123456789,KH987654321.
- Se desplegarán todas las peticiones que tengan asociado los IDs indicados.
- No es necesario indicar la fecha, ya que los Request IDs son identificadores únicos de una solicitud de despliegue.



El log de salida del despliegue se verá de la siguiente forma:

Task Response - Check_exec_deploy_job success

NS: gl.ether.devops Job Name: deploy-gl-apx-app-by-request-id

deploy success

Type: Service

Go

light

Exit Status: 0

=====

Starting deployment by request ID KH09102020114620995943

- a2s3-deploy execution started.

INFO - Logging deploy action with "b154f030-4608-4ced-bf59-6ae0edd2f138" trace id

INFO - Notifing start deploy to Kharon

Procedimiento de Contingencia al frontal de Cronos para despliegues

Ante una caída o indisponibilidad puntual del propio frontal de la herramienta, se detalla un proceso de contingencia para que los equipos que necesiten realizar algún despliegue puedan seguir haciéndolo de forma autónoma y sin necesidad de recurrir al soporte 24x7 vía SALA.

- Documentación base de Cronos para uso por CLI
 - <https://platform.bbva.com/en-us/disciplines-governance/disciplines/iaas/documentati on/cronos/plan-contingencia/front-indisponible>
- Documentación específica para ejecución de jobs Cronos de despliegue por CLI
 - https://docs.google.com/document/d/1OR-UHsqCmaFIMTM7UYdR-P7aDsxq_s_KrBYgWrklGj4/edit?usp=sharing

Rollback de un despliegue en Producción

La funcionalidad de rollback nos permite dar marcha atrás la release version activa de un recurso en Producción. Para ejecutar el rollback de un despliegue, se debe seleccionar “Deployment rollback” desde la Consola y automáticamente se realizará el despliegue de la anterior versión activa.

The screenshot displays a deployment management interface. At the top, there are buttons for 'CREATE RELEASE VERSION...', 'CREATE BUGFIX...', and 'DEPLOY...'. Below these, a table lists versions across five environments: DEV, INT, AUS, OCT, and PRO. The versions listed are:

- BRANCH** `desp_hotfix-SNAPSHOT` (hotfix/test_deploy_hotfix)
- TAG** `0.4.1-RELEASE` (0.4.1-RELEASE)
- TAG** `0.4.0-RELEASE` (0.4.0-RELEASE) - This version is marked as **ACTIVE** in all environments.
- BRANCH** `prueba-qwput850-SNAPSHOT` (feature/APX1-2731-revisar-transaccion-de-prueba-qwput850)

A context menu is open over the 'PRO' column for the '0.4.0-RELEASE' row, showing the following options:

- Redeploy
- Deployment Rollback** (highlighted with a red box)
- See deployment audits

Antes de hacer rollback, es importante tener en cuenta algunas consideraciones si se trata de un primer despliegue en Consola o si se trata de un recurso de tipo librería. Veremos el detalle en los siguientes apartados:

Rollback del primer despliegue en Consola Ether

Para poder ejecutar el rollback de forma satisfactoria, debe existir un despliegue anterior de nuestro recurso que poder restaurar, de lo contrario el rollback fallará. En el caso de que exista un despliegue anterior realizado desde el Jenkins de LCM, el rollback automáticamente restaurará este despliegue de forma transparente para el usuario.

Rollback del despliegue de un recurso tipo librería

Los recursos de tipo librería tienen una particularidad frente al resto de recursos en APX: pueden ser desplegados en modo Online, Batch o Multiruntime (Online+Batch). A la hora de realizar el rollback de una librería, es fundamental que exista correspondencia en el modo de despliegue (Online, Batch o Multiruntime) con el despliegue anterior. Solo en caso de darse esta correspondencia funcionará correctamente el rollback automático. En caso contrario, al hacer rollback se producirá un error y se mostrará un mensaje informativo indicando que no es posible hacer un rollback automático y deberá ser el developer quien restaure el entorno manualmente realizando **rollforward**, bien redesplegando alguna versión anterior o bien generando una nueva versión y desplegando por entornos previos hasta Producción. Veamos algunos casos concretos:

1. Rollback de librería desplegada en modo multiruntime

Este caso aplica a librerías desplegadas en Online y Batch con la nueva solución de despliegue multiruntime (en Producción desde el 14/12/2022) . Al hacer un rollback de este tipo de despliegue, se restaurará automáticamente el despliegue anterior de forma simultánea en Online y Batch siempre que haya correspondencia en el modo de despliegue, es decir, el despliegue anterior también debe ser multiruntime (Online&Batch). Veámos un par de ejemplos:

Ejemplo 1: Tenemos la versión V2 de una librería desplegada hasta producción en modo multiruntime (Online&Batch) y una versión anterior V1 también desplegada en modo multiruntime:

VERSIÓN	DEV	INT	PRO
V2	Online&Batch (ACTIVA)	Online&Batch (ACTIVA)	Online&Batch (ACTIVA)
V1	Online&Batch (Superseded)	Online&Batch (Superseded)	Online&Batch (Superseded)

Si hacemos rollback en Producción de la versión V2, se restaurará correctamente la versión V1 en Online y Batch ya que, en este caso, existe correspondencia de runtimes con el despliegue anterior:

VERSIÓN	DEV	INT	PRO
V2	Online&Batch (ACTIVA)	Online&Batch (ACTIVA)	Online&Batch (Superseded)
V1	Online&Batch (Superseded)	Online&Batch (Superseded)	Online&Batch (ACTIVA)

Ejemplo 2: Tenemos la versión V2 de una librería desplegada hasta producción en modo multiruntime (Online&Batch) y una versión anterior V1 solo desplegada en modo online:

VERSIÓN	DEV	INT	PRO
V2	Online&Batch (ACTIVA)	Online&Batch (ACTIVA)	Online&Batch (ACTIVA)
V1	Online (Superseded)	Online (Superseded)	Online (Superseded)

Si hacemos rollback en Producción de la versión V2, fallará porque no existe correspondencia de runtimes, es decir, la versión V1 solo está desplegada en Online y no en modo multiruntime.

VERSIÓN	DEV	INT	PRO
V2	Online&Batch (ACTIVA)	Online&Batch (ACTIVA)	Online&Batch (ACTIVA)
V1	Online (Superseded)	Online (Superseded)	Online (Superseded)

En este caso, la forma de proceder para restaurar el entorno, es hacer **rollforward**. Una posibilidad sería redespregar la versión V1 desde el primer entorno hasta PR, teniendo en cuenta que al tratarse ahora de un despliegue multiruntime, la versión V1 desplegaría tanto en Online como en Batch

VERSIÓN	DEV	INT	PRO
V2	Online&Batch (Superseded)	Online&Batch (Superseded)	Online&Batch (Superseded)
V1	Online&Batch (ACTIVA)	Online&Batch (ACTIVA)	Online&Batch (ACTIVA)

2. Rollback de librería desplegada en modo monoruntime (Online o Batch)

Este caso aplica a librerías desplegadas en un único runtime: Online o Batch. Al hacer un rollback de este tipo de despliegue, se restaurará automáticamente el despliegue anterior siempre que haya correspondencia en el modo de despliegue, es decir, el despliegue anterior también debe ser Online o Batch según corresponda. Veámos un par de ejemplos:

Ejemplo 1: Tenemos la versión V2 de una librería desplegada hasta producción en modo Online y una versión anterior de la librería V1 desplegada también en Online:

VERSIÓN	DEV	INT	PRO
V2	Online (ACTIVA)	Online (ACTIVA)	Online (ACTIVA)
V1	Online (Superseded)	Online (Superseded)	Online (Superseded)

Si hacemos rollback en Producción de la versión V2, se restaurará correctamente la versión V1 ya que, en este caso, existe correspondencia de runtimes con el despliegue anterior (Online):

VERSIÓN	DEV	INT	PRO
V2	Online (ACTIVA)	Online (ACTIVA)	Online (Superseded)
V1	Online (Superseded)	Online (Superseded)	Online (ACTIVA)

Ejemplo 2: Tenemos la versión V2 de una librería desplegada hasta producción en modo Online y una versión anterior de la librería V2 desplegada en Batch:

VERSIÓN	DEV	INT	PRO
V2	Online (ACTIVA)	Online (ACTIVA)	Online (ACTIVA)
V1	Batch (Superseded)	Batch (Superseded)	Batch (Superseded)

Si hacemos rollback en Producción de la versión V2, fallará porque no existe correspondencia de runtimes, es decir, la versión V1 está desplegada en modo Batch y no en modo Online.

VERSIÓN	DEV	INT	PRO
V2	Online (ACTIVA)	Online (ACTIVA)	Online (ACTIVA)
V1	Batch (Superseded)	Batch (Superseded)	Batch (Superseded)

En este caso, la forma de proceder para restaurar el entorno, es hacer **rollforward**. Una posibilidad sería redespregar la versión V1 desde el primer entorno hasta PR. Hay que tener en cuenta que al tratarse ahora de un despliegue de tipo Online, la versión V1 desplegará en Online y no en Batch:

VERSIÓN	DEV	INT	PRO
V2	Online (Superseded)	Online (Superseded)	Online (Superseded)
V1	Online (ACTIVA)	Online (ACTIVA)	Online (ACTIVA)

3. Rollback de librería desplegada por primera vez en Consola

Este caso aplica a librerías desplegadas por primera vez en Consola y cuyo despliegue anterior fue realizado con el modelo legacy LCM. Al hacer un rollback de este tipo de despliegue, se restaurará automáticamente el despliegue anterior siempre que haya correspondencia en el modo de despliegue, es decir, el despliegue anterior realizado con el modelo LCM también debe ser Online, Batch u Online&Batch según corresponda. Veámos un par de ejemplos:

Ejemplo 1: Tenemos la versión V1 de una librería desplegada hasta producción en modo multiruntime (Online&Batch) y una versión V0 desplegada con el modelo Legacy tanto en Online como en Batch:

VERSIÓN	DEV	INT	PRO
V1	Online&Batch (ACTIVA)	Online&Batch (ACTIVA)	Online&Batch (ACTIVA)

Si hacemos rollback en Producción de la versión V1, se restaurará correctamente la versión V0 en Online y Batch ya que, en este caso, existe correspondencia de runtimes con los despliegues anteriores.

Ejemplo 2: Tenemos la versión V1 de una librería desplegada hasta producción en modo multiruntime (Online&Batch) y una versión V0 desplegada con el modelo Legacy solo en Online:

VERSIÓN	DEV	INT	PRO
V1	Online&Batch (ACTIVA)	Online&Batch (ACTIVA)	Online&Batch (ACTIVA)

Si hacemos rollback en Producción de la versión V1, el rollback automático fallará ya que no se da correspondencia de runtimes con la versión V0 y la única forma de restaurar el entorno es hacer **rollforward**. En este caso, como el despliegue de la V0 se realizó con el sistema legacy, no tenemos forma en Consola de hacer un rollforward directo. Una forma de actuar sería

generar una nueva versión a partir del commit que fue desplegado en Producción con el sistema Legacy. Para ello, habría que seguir los siguientes pasos:

- Nos situamos en el primer commit de la migración de nuestro componente:

bot-apx-stacker	f8d9ce9b658	***Migration commit*** - created develop branch	Este NO
bot-apx-stacker	5ffd03d3fa2	***Migration commit*** from Stash	Este commit

- Entramos en el pom.xml de la raíz del repositorio e identificamos la versión que tiene nuestro componente en ese commit:

Source

🔗 master ▾ ... kyfec001 / pom.xml

Source view | Diff to previous | History ▾ | 3.2 KB | Contributors ▾

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.
2   <modelVersion>4.0.0</modelVersion>
3   <parent>
4     <artifactId>elara-project</artifactId>
5     <groupId>com.bbva.elara</groupId>
6     <version>7.1.7</version>
7   </parent>
8
9   <groupId>com.bbva.kyfe.dto.template</groupId>
10  <artifactId>KYFEC001</artifactId>
11  <version>0.0.1</version>
12  <packaging>bundle</packaging>
13  <description>Contiene BBVATemplate</description>
14  <properties>
15
```

- En este caso sería la versión 0.0.1. Con el mayor y el menor de la versión, en este caso '0.0', creamos una rama llamada 'release/[major].[minor]'. En este caso se crearía la rama 'release/0.0'.
- Esperamos a que pase el pipeline de la nueva rama release y cuando termine nos creará la Release Version en consola y ya estará listo para desplegar.

Rollback del primer despliegue de un recurso Shell Script migrado a GitHub

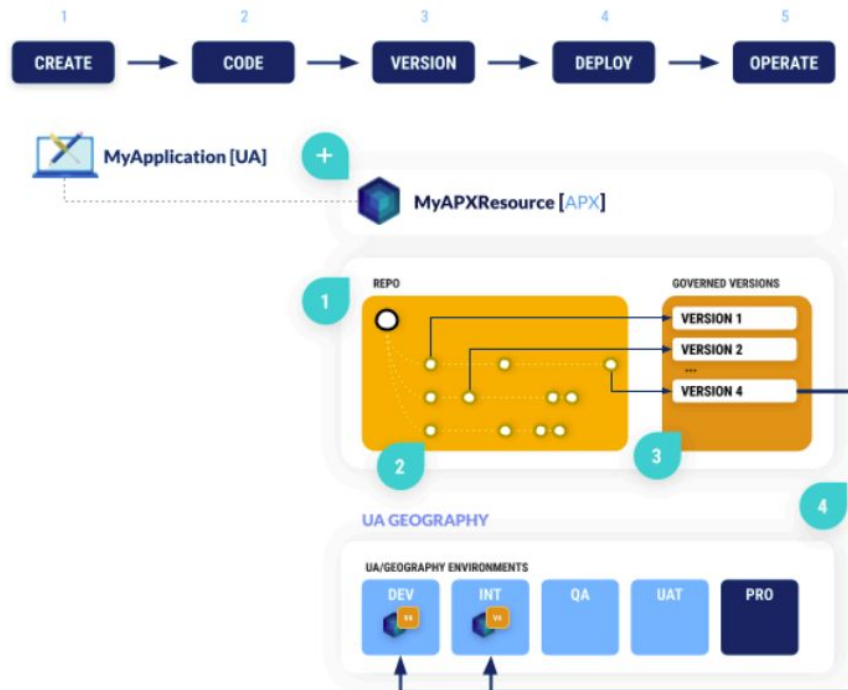
Para los recursos de tipo **Shell Script** cuyos repositorios hayan sido migrados a Github es importante tener en cuenta que una vez hecha la migración, no será posible desplegar versiones anteriores generadas con el pipeline legacy de Jenkins. Al intentar hacer un despliegue de este tipo se pintará un error en Consola con esta información y la manera de proceder. La solución pasa por generar una nueva versión en Github a partir del commit que se necesite desplegar.

De la misma forma, si se intenta hacer un **rollback** automático en Producción de una versión legacy, se obtendrá el mismo error y habrá que proceder de la misma manera, generando una nueva versión en Github y desplegando por todos los entornos hasta Producción.

Políticas de Gobierno en versiones de un componente

Como parte del ciclo de vida estándar de los recursos técnicos que forma parte de una Ether Application (como es el caso de los recursos de tipo APX), se encuentra la fase de declarar y configurar versiones de los mismos, para poder desplegarlas por los distintos entornos.

TECHNICAL RESOURCE LIFECYCLE



- 1] Creación del recurso. En este punto se asocia el nuevo recurso a la UA en la Consola, y se crea y configura el nuevo repositorio donde se implementará el código del componente.
- 2] En la segunda etapa, el equipo de desarrollo implementa (por fuera de la Consola), el código que requiera el componente. Realizando los commits, merges, PRs, etc. siguiendo las directrices y el Branching Model a aplicar.
- 3] Antes de poder desplegar el recurso en cualquier de los entornos de la aplicación, se debe crear desde la Consola una versión del componente. Parte de la configuración de esta versión que deberá indicar el usuario, será: darle un nombre a la versión, indicar el origen del código fuente: Tag, commit, rama (siendo en este último caso una versión de tipo snapshot que en cada despliegue de la versión desplegará el commit al que esté apuntando la rama en ese momento).

Como parte de la información o metadatos que se almacenarán en la Consola como parte de la versión, serán los tickets JIRA que las políticas de Gobierno han mandado como requeridos de cara a tener trazabilidad sobre el componente implementado y sus despliegues.

- 4] En el momento del despliegue, de una versión del componente, y en concreto en alguno de los entornos/namespaces de una geografía de la aplicación, es cuando la plataforma valida que el ticket JIRA asociado (específico por geografía), se encuentra en el estado adecuado para permitir dicho despliegue. **Disclaimer:** Para la geografía México no es requerido ingresar un ticket JIRA, es opcional, pero si se cuenta con uno se puede ingresar aún siendo de tipo **MP-**.

Seguridad

Como parte del ciclo de desarrollo de software (SDLC, Software Development Life Cycle), la seguridad, al igual que la calidad, es uno de los fundamentos a tener presente desde los primeros estadios del desarrollo (SSDLC, Secure Software Development Life Cycle).

Para poder conocer el nivel de seguridad de los desarrollos en los que cualquier usuario trabaja, se cuenta con una herramienta en la casa llamada Chimera.

Chimera ofrece feedback de seguridad en cualquier estado del ciclo de vida del desarrollo, incluso en estados tempranos o iniciales del mismo. Es el concepto o lo que se conoce como “*shift left*” security y que persigue “mover” la seguridad al estado del desarrollo lo más temprano posible para reducir los posibles riesgos de seguridad y mitigar y resolver los existentes lo antes posible.

Chimera ofrece a día de hoy 3 funcionalidades en cuanto a feedback de seguridad:

- Code Review o SAST (Static Application Security Testing). Revisión del estado de seguridad de código o búsqueda de vulnerabilidades conocidas en código.
- Image Review. Revisión del estado de seguridad de imágenes docker o búsqueda de vulnerabilidades conocidas en imágenes. NOTA: esta funcionalidad no está aún industrializada.
- Secrets Review. Búsqueda de secretos en repositorios. NOTA: esta funcionalidad no está aún industrializada.

Para conocer estas funcionalidades en mayor profundidad, por favor, acudir al documento propio de Chimera. NOTA: este documento está aún en fase de construcción.

Al igual que con Sonar un proyecto puede ver el resultado o informe del análisis ejecutado, un proyecto puede ver el resultado o informe del análisis de seguridad del mismo llevado a cabo por Chimera.

El resultado del análisis o informe, también conocido como *review*, está disponible desde la propia herramienta (Chimera) así como desde el correspondiente comportamiento de SAMUEL que chequea el resultado de la valoración del análisis de seguridad.

El comportamiento se llama *chimera-code-review* y con él se chequea si el repositorio cumple o no con los criterios de seguridad establecidos en Chimera y evaluados por Chimera. Esos criterios de seguridad son lo que se conoce en Chimera como *política* y la definición de la misma puede variar en base a 6 parámetros de catalogación de los repositorios:

- **Product.** Producto de la aplicación al que corresponde el repositorio. P.ej: 'tarjetasempresa' de la aplicación NetCash España
- **Area.** Área, departamento o estructura organizativa a la que pertenece el repositorio. P.ej: Enterprise Client Systems
- **Country.** Geografía de implementación (que no despliegue). P.ej: españa
- **Architecture.** Arquitectura o tecnología del repositorio. P.ej: spring
- **Language.** Lenguaje de programación del repositorio. P.ej: java
- **UUAA.** Unidad applicativa a la que pertenece el repositorio. P.ej: ebcf

En el caso de que el proyecto cumpla con la política, el estado del comportamiento será “passed” (verde), mientras que si el proyecto no cumple, el estado será “not passed” (rojo) cuando el enforcement esté activo. Si el enforcement no está activo, el estado será “not passed but not blocking”.

Dado que el análisis de seguridad es **asíncrono** a la ejecución de los pipelines, aunque el análisis se haya solicitado ya a Chimera, el estado del comportamiento será “not-evaluated yet” (PENDING) (naranja) mientras que el análisis no se complete por parte de Chimera. SAM[uel] y Chimera tienen una integración en la que Chimera notifica del identificador único del análisis (scan id) de forma que SAM[uel] pueda acudir a chequear el resultado de dicho análisis (scan) cuando éste sea notificado por Chimera. Una vez el scan/análisis se haya completado, el resultado del mismo (la *review*) estará disponible como para ser chequeado por el comportamiento de SAM[uel] y el estado del comportamiento se actualizará en consonancia.

Todo esto se explica a través de un diagrama ilustrativo y descripción textual en el apartado “**Chimera pipeline integration via Samuel**” del propio documento de Chimera. NOTA: este documento está aún en fase de construcción.